

Module 9: Machine Learning II

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

- Advanced machine learning algorithms
- Ensemble methods
- Unsupervised learning techniques
- Model optimization and tuning
- Real-world ML project workflow

Topics Covered

1. Random Forest and ensemble methods
2. Support Vector Machines (SVM)
3. K-Nearest Neighbors (KNN)
4. Clustering algorithms (K-means, hierarchical)
5. Dimensionality reduction (PCA)
6. Model hyperparameter tuning
7. Cross-validation strategies
8. Model selection and comparison
9. ML pipeline creation

Exercises

- Advanced ML algorithm implementation
- Complete ML project workflow

1. Random Forest and Ensemble Methods

What is Random Forest?

Random Forest is an ensemble learning method that combines multiple decision trees to make more accurate predictions. It's like asking many experts for their opinion and taking the majority vote.

Key Concepts:

- **Bootstrap Aggregating (Bagging):** Each tree is trained on a random subset of data

- **Feature Randomness:** Each tree considers only a random subset of features
- **Voting:** Final prediction is the majority vote (classification) or average (regression)
- **Out-of-bag (OOB) Error:** Built-in validation using samples not used in training

Advantages:

- Reduces overfitting
- Handles missing values well
- Works with both numerical and categorical data
- Provides feature importance scores
- Works well out-of-the-box

In [...

```

1  # Random Forest and Ensemble Methods - Simplified Examples
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from sklearn.datasets import make_classification, load_breast_cancer
6  from sklearn.model_selection import train_test_split, cross_val_score
7  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
8  from sklearn.tree import DecisionTreeClassifier
9  from sklearn.metrics import classification_report, confusion_matrix
10 import warnings
11 warnings.filterwarnings('ignore')
12
13
14 # Create a simple dataset for demonstration
15 print("=== CREATING SIMPLE DATASET ===")
16 X, y = make_classification(n_samples=1000, n_features=4, n_redundant=0,
17                           n_clusters_per_class=1, random_state=42)
18 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
19                                                     random_state=42)
20
21 print(f"Dataset shape: {X.shape}")
22 print(f"Classes: {np.unique(y)}")
23 print(f"Class distribution: {np.bincount(y)}")
24
25 # 1. Single Decision Tree (for comparison)
26 print("\n=== SINGLE DECISION TREE ===")
27 tree = DecisionTreeClassifier(random_state=42)
28 tree.fit(X_train, y_train)
29 tree_pred = tree.predict(X_test)
30 tree_accuracy = accuracy_score(y_test, tree_pred)
31 print(f"Single Tree Accuracy: {tree_accuracy:.3f}")
32
33 # 2. Random Forest - Basic Example
34 print("\n=== RANDOM FOREST - BASIC EXAMPLE ===")
35 rf = RandomForestClassifier(n_estimators=100, random_state=42)
36 rf.fit(X_train, y_train)
37 rf_pred = rf.predict(X_test)
38 rf_accuracy = accuracy_score(y_test, rf_pred)
39 print(f"Random Forest Accuracy: {rf_accuracy:.3f}")
40
41 # Feature importance
42 print(f"Feature importance: {rf.feature_importances_}")
43
44 # 3. Random Forest - Understanding Parameters
45 print("\n=== RANDOM FOREST - PARAMETER UNDERSTANDING ===")
46

```

```

47
48 # Test different number of trees
49 n_estimators_list = [10, 50, 100, 200, 500]
50 for n_est in n_estimators_list:
51     rf_temp = RandomForestClassifier(n_estimators=n_est, random_s
52     rf_temp.fit(X_train, y_train)
53     score = rf_temp.score(X_test, y_test)
54     print(f"n_estimators={n_est}: Accuracy={score:.3f}")
55
56 # 4. Random Forest - Out-of-Bag Score
57 print("\n=== RANDOM FOREST - OUT-OF-BAG SCORE ===")
58 rf_oob = RandomForestClassifier(n_estimators=100, oob_score=True,
59 rf_oob.fit(X_train, y_train)
60 print(f"Out-of-bag score: {rf_oob.oob_score_:.3f}")
61 print(f"Test accuracy: {rf_oob.score(X_test, y_test):.3f}")
62
63 # 5. Gradient Boosting
64 print("\n=== GRADIENT BOOSTING ===")
65 gb = GradientBoostingClassifier(n_estimators=100, learning_rate=0
66 gb.fit(X_train, y_train)
67 gb_pred = gb.predict(X_test)
68 gb_accuracy = accuracy_score(y_test, gb_pred)
69 print(f"Gradient Boosting Accuracy: {gb_accuracy:.3f}")
70
71 # 6. Bagging (Bootstrap Aggregating)
72 print("\n=== BAGGING ===")
73 bagging = BaggingClassifier(
74     base_estimator=DecisionTreeClassifier(),
75     n_estimators=100,
76     random_state=42
77 )
78 bagging.fit(X_train, y_train)
79 bagging_pred = bagging.predict(X_test)
80 bagging_accuracy = accuracy_score(y_test, bagging_pred)
81 print(f"Bagging Accuracy: {bagging_accuracy:.3f}")
82
83 # 7. Voting Classifier (Ensemble)
84 print("\n=== VOTING CLASSIFIER ===")
85 # Create individual classifiers
86 rf_vote = RandomForestClassifier(n_estimators=100, random_state=4
87 gb_vote = GradientBoostingClassifier(n_estimators=100, random_sta
88 tree_vote = DecisionTreeClassifier(random_state=42)
89
90 # Hard voting (majority vote)
91 voting_hard = VotingClassifier(
92     estimators=[('rf', rf_vote), ('gb', gb_vote), ('tree', tree_v
93     voting='hard'
94 )
95 voting_hard.fit(X_train, y_train)
96 hard_pred = voting_hard.predict(X_test)
97 hard_accuracy = accuracy_score(y_test, hard_pred)
98 print(f"Hard Voting Accuracy: {hard_accuracy:.3f}")
99
100 # Soft voting (average probabilities)
101 voting_soft = VotingClassifier(
102     estimators=[('rf', rf_vote), ('gb', gb_vote), ('tree', tree_v
103     voting='soft'
104 )
105 voting_soft.fit(X_train, y_train)
106 soft_pred = voting_soft.predict(X_test)
107
108
109
110

```

```

111 soft_accuracy = accuracy_score(y_test, soft_pred)
112 print(f"Soft Voting Accuracy: {soft_accuracy:.3f}")
113
114 # 8. Model Comparison
115 print("\n=== MODEL COMPARISON ===")
116 models = {
117     'Single Tree': tree,
118     'Random Forest': rf,
119     'Gradient Boosting': gb,
120     'Bagging': bagging,
121     'Hard Voting': voting_hard,
122     'Soft Voting': voting_soft
123 }
124
125
126 results = []
127 for name, model in models.items():
128     accuracy = model.score(X_test, y_test)
129     results.append({'Model': name, 'Accuracy': accuracy})
130     print(f"{name}: {accuracy:.3f}")
131
132 results_df = pd.DataFrame(results)
133 print(f"\nResults Summary:")
134 print(results_df)
135
136 # 9. Feature Importance Visualization
137 print("\n=== FEATURE IMPORTANCE VISUALIZATION ===")
138 feature_names = [f'Feature_{i}' for i in range(X.shape[1])]
139 importance_df = pd.DataFrame({
140     'feature': feature_names,
141     'importance': rf.feature_importances_
142 }).sort_values('importance', ascending=False)
143
144
145 print("Feature Importance:")
146 print(importance_df)
147
148 # Plot feature importance
149 plt.figure(figsize=(10, 6))
150 plt.bar(importance_df['feature'], importance_df['importance'])
151 plt.title('Random Forest Feature Importance')
152 plt.xlabel('Features')
153 plt.ylabel('Importance')
154 plt.xticks(rotation=45)
155 plt.tight_layout()
156 plt.show()
157
158 # 10. Learning Curves
159 print("\n=== LEARNING CURVES ===")
160 # Test different training set sizes
161 train_sizes = [50, 100, 200, 300, 400, 500, 600, 700, 800]
162 train_scores = []
163 test_scores = []
164
165
166 for size in train_sizes:
167     # Sample data
168     indices = np.random.choice(len(X_train), size=size, replace=False)
169     X_sample = X_train[indices]
170     y_sample = y_train[indices]
171
172     # Train model
173     rf_sample = RandomForestClassifier(n_estimators=100, random_s

```

```

175         rf_sample.fit(X_sample, y_sample)
176
177     # Calculate scores
178     train_score = rf_sample.score(X_sample, y_sample)
179     test_score = rf_sample.score(X_test, y_test)
180
181     train_scores.append(train_score)
182     test_scores.append(test_score)
183
184 # Plot learning curves
185 plt.figure(figsize=(10, 6))
186 plt.plot(train_sizes, train_scores, 'o-', label='Training Score')
187 plt.plot(train_sizes, test_scores, 'o-', label='Test Score')
188 plt.xlabel('Training Set Size')
189 plt.ylabel('Accuracy')
190 plt.title('Random Forest Learning Curves')
191 plt.legend()
192 plt.grid(True)
193 plt.show()
194
195 # 11. Hyperparameter Tuning
196 print("\n=== HYPERPARAMETER TUNING ===")
197 param_grid = {
198     'n_estimators': [50, 100, 200],
199     'max_depth': [None, 10, 20],
200     'min_samples_split': [2, 5, 10]
201 }
202
203
204
205 grid_search = GridSearchCV(
206     RandomForestClassifier(random_state=42),
207     param_grid,
208     cv=5,
209     scoring='accuracy',
210     n_jobs=-1
211 )
212
213 grid_search.fit(X_train, y_train)
214 print(f"Best parameters: {grid_search.best_params_}")
215 print(f"Best cross-validation score: {grid_search.best_score_: .3f}")
216 print(f"Test accuracy with best params: {grid_search.score(X_test, y_test): .3f}")
217
218 # 12. Real-world Example - Breast Cancer Dataset
219 print("\n=== REAL-WORLD EXAMPLE - BREAST CANCER DATASET ===")
220 cancer = load_breast_cancer()
221 X_cancer, y_cancer = cancer.data, cancer.target
222 X_cancer_train, X_cancer_test, y_cancer_train, y_cancer_test = train_test_split(
223     X_cancer, y_cancer, test_size=0.2, random_state=42)
224
225
226
227 # Train Random Forest on real data
228 rf_cancer = RandomForestClassifier(n_estimators=100, random_state=42)
229 rf_cancer.fit(X_cancer_train, y_cancer_train)
230 cancer_pred = rf_cancer.predict(X_cancer_test)
231 cancer_accuracy = accuracy_score(y_cancer_test, cancer_pred)
232
233 print(f"Breast Cancer Dataset - Random Forest Accuracy: {cancer_accuracy: .3f}")
234 print(f"Feature importance (top 5):")
235 feature_importance = pd.DataFrame({
236     'feature': cancer.feature_names,
237     'importance': rf_cancer.feature_importances_
238 })

```

```

239     }).sort_values('importance', ascending=False).head()
240
    print(feature_importance)

```

13. Summary

```

print("\n=== RANDOM FOREST SUMMARY ===")
print("1. Random Forest combines multiple decision trees")
print("2. Each tree is trained on a random subset of data and fea")
print("3. Final prediction is the majority vote (classification)")
print("4. Reduces overfitting compared to single decision tree")
print("5. Provides feature importance scores")
print("6. Works well out-of-the-box with default parameters")
print("7. Can handle missing values and mixed data types")
print("8. Ensemble methods (voting, bagging) often improve perfor")
print("9. Hyperparameter tuning can further improve performance")
print("10. Cross-validation provides robust performance estimates")

```

2. Support Vector Machines (SVM)

What is SVM?

Support Vector Machine is a powerful classification algorithm that finds the best boundary (hyperplane) to separate different classes. It's like finding the best line to separate two groups of points.

Key Concepts:

- **Support Vectors:** Data points closest to the decision boundary
- **Margin:** Distance between the decision boundary and the nearest data points
- **Kernel Trick:** Transforms data into higher dimensions to find non-linear boundaries
- **C Parameter:** Controls the trade-off between margin maximization and error minimization

Types of Kernels:

- **Linear:** For linearly separable data
- **RBF (Radial Basis Function):** For non-linear data, most commonly used
- **Polynomial:** For polynomial relationships
- **Sigmoid:** For neural network-like behavior

Advantages:

- Works well with high-dimensional data
- Memory efficient (only uses support vectors)
- Versatile (different kernels for different data types)
- Good generalization performance

In [...

```
1  # Support Vector Machines (SVM) - Simplified Examples
2
3  import pandas as pd
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import make_classification, make_circles, make_moons
7  from sklearn.model_selection import train_test_split, cross_val_score
8  from sklearn.svm import SVC
9  from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import classification_report, confusion_matrix
11 import warnings
12 warnings.filterwarnings('ignore')
13
14 # Create different types of datasets to demonstrate SVM capabilities
15 print("=== CREATING DIFFERENT DATASET TYPES ===")
16
17 # Dataset 1: Linearly separable data
18 X1, y1 = make_classification(n_samples=300, n_features=2, n_redundant=0,
19                             n_clusters_per_class=1, random_state=42)
20
21 # Dataset 2: Non-linearly separable data (circles)
22 X2, y2 = make_circles(n_samples=300, noise=0.1, factor=0.3, random_state=42)
23
24 # Dataset 3: Non-linearly separable data (moons)
25 X3, y3 = make_moons(n_samples=300, noise=0.1, random_state=42)
26
27 # Dataset 4: High-dimensional data
28 X4, y4 = make_classification(n_samples=1000, n_features=20, n_redundant=5,
29                             n_clusters_per_class=1, random_state=42)
30
31 print(f"Dataset 1 (Linear): {X1.shape}, classes: {np.unique(y1)}")
32 print(f"Dataset 2 (Circles): {X2.shape}, classes: {np.unique(y2)}")
33 print(f"Dataset 3 (Moons): {X3.shape}, classes: {np.unique(y3)}")
34 print(f"Dataset 4 (High-dim): {X4.shape}, classes: {np.unique(y4)}")
35
36 # Split datasets
37
38 X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2,
39                                                         random_state=42)
40 X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2,
41                                                         random_state=42)
42 X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size=0.2,
43                                                         random_state=42)
44 X4_train, X4_test, y4_train, y4_test = train_test_split(X4, y4, test_size=0.2,
45                                                         random_state=42)
46
47 # 1. Linear SVM
48 print("\n=== LINEAR SVM ===")
49 svm_linear = SVC(kernel='linear', random_state=42)
50 svm_linear.fit(X1_train, y1_train)
51 linear_pred = svm_linear.predict(X1_test)
52 linear_accuracy = accuracy_score(y1_test, linear_pred)
53 print(f"Linear SVM Accuracy: {linear_accuracy:.3f}")
54
55 # 2. RBF (Radial Basis Function) SVM
56 print("\n=== RBF SVM ===")
57 svm_rbf = SVC(kernel='rbf', random_state=42)
58 svm_rbf.fit(X2_train, y2_train)
59 rbf_pred = svm_rbf.predict(X2_test)
60 rbf_accuracy = accuracy_score(y2_test, rbf_pred)
61 print(f"RBF SVM Accuracy: {rbf_accuracy:.3f}")
62
63 # 3. Polynomial SVM
64 print("\n=== POLYNOMIAL SVM ===")
```

```

63 svm_poly = SVC(kernel='poly', degree=3, random_state=42)
64 svm_poly.fit(X3_train, y3_train)
65 poly_pred = svm_poly.predict(X3_test)
66 poly_accuracy = accuracy_score(y3_test, poly_pred)
67 print(f"Polynomial SVM Accuracy: {poly_accuracy:.3f}")
68
69 # 4. Understanding C Parameter
70 print("\n=== UNDERSTANDING C PARAMETER ===")
71 # C controls the trade-off between margin maximization and error
72 # Higher C = less tolerance for errors, more complex boundary
73 # Lower C = more tolerance for errors, simpler boundary
74
75 C_values = [0.1, 1, 10, 100, 1000]
76 for C in C_values:
77     svm_temp = SVC(kernel='rbf', C=C, random_state=42)
78     svm_temp.fit(X2_train, y2_train)
79     score = svm_temp.score(X2_test, y2_test)
80     print(f"C={C}: Accuracy={score:.3f}")
81
82 # 5. Understanding Gamma Parameter (for RBF kernel)
83 print("\n=== UNDERSTANDING GAMMA PARAMETER ===")
84 # Gamma controls the influence of individual training examples
85 # Higher gamma = more influence of nearby points, more complex bo
86 # Lower gamma = less influence, simpler boundary
87
88 gamma_values = [0.001, 0.01, 0.1, 1, 10]
89 for gamma in gamma_values:
90     svm_temp = SVC(kernel='rbf', gamma=gamma, random_state=42)
91     svm_temp.fit(X2_train, y2_train)
92     score = svm_temp.score(X2_test, y2_test)
93     print(f"Gamma={gamma}: Accuracy={score:.3f}")
94
95 # 6. SVM with Feature Scaling
96 print("\n=== SVM WITH FEATURE SCALING ===")
97 # SVM is sensitive to feature scaling, so we need to scale the da
98
99 # Without scaling
100 svm_no_scale = SVC(kernel='rbf', random_state=42)
101 svm_no_scale.fit(X4_train, y4_train)
102 no_scale_accuracy = svm_no_scale.score(X4_test, y4_test)
103 print(f"Without scaling: {no_scale_accuracy:.3f}")
104
105 # With scaling
106 scaler = StandardScaler()
107 X4_train_scaled = scaler.fit_transform(X4_train)
108 X4_test_scaled = scaler.transform(X4_test)
109
110 svm_scaled = SVC(kernel='rbf', random_state=42)
111 svm_scaled.fit(X4_train_scaled, y4_train)
112 scaled_accuracy = svm_scaled.score(X4_test_scaled, y4_test)
113 print(f"With scaling: {scaled_accuracy:.3f}")
114
115 # 7. Hyperparameter Tuning
116 print("\n=== HYPERPARAMETER TUNING ===")
117 param_grid = {
118     'C': [0.1, 1, 10, 100],
119     'gamma': [0.001, 0.01, 0.1, 1],
120     'kernel': ['rbf', 'poly', 'sigmoid']
121 }
122
123
124
125
126

```



```

127 grid_search = GridSearchCV(
128     SVC(random_state=42),
129     param_grid,
130     cv=5,
131     scoring='accuracy',
132     n_jobs=-1
133 )
134
135 grid_search.fit(X4_train_scaled, y4_train)
136 print(f"Best parameters: {grid_search.best_params_}")
137 print(f"Best cross-validation score: {grid_search.best_score_:.3f}")
138 print(f"Test accuracy with best params: {grid_search.score(X4_test_scaled, y4_test)}")
139
140
141 # 8. Support Vectors
142 print("\n=== SUPPORT VECTORS ===")
143 # Support vectors are the data points that define the decision boundary
144 svm_support = SVC(kernel='rbf', random_state=42)
145 svm_support.fit(X2_train, y2_train)
146
147 print(f"Number of support vectors: {svm_support.n_support_}")
148 print(f"Total support vectors: {svm_support.n_support_.sum()}")
149 print(f"Support vector indices: {svm_support.support_}")
150
151 # 9. Decision Boundary Visualization
152 print("\n=== DECISION BOUNDARY VISUALIZATION ===")
153 # Create a mesh grid for visualization
154 def plot_decision_boundary(X, y, model, title):
155     h = 0.02
156     x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
157     y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
158     xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
159                          np.arange(y_min, y_max, h))
160
161     Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
162     Z = Z.reshape(xx.shape)
163
164     plt.figure(figsize=(8, 6))
165     plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.RdYlBu)
166     plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolor='k')
167     plt.title(title)
168     plt.xlabel('Feature 1')
169     plt.ylabel('Feature 2')
170     plt.show()
171
172
173 # Plot decision boundaries for different kernels
174 plot_decision_boundary(X2, y2, svm_rbf, 'RBF SVM Decision Boundary')
175
176
177 # 10. Cross-Validation
178 print("\n=== CROSS-VALIDATION ===")
179 cv_scores = cross_val_score(SVC(kernel='rbf', random_state=42), X2, y2, cv=5)
180 print(f"Cross-validation scores: {cv_scores}")
181 print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std()})")
182
183 # 11. Real-world Example - Breast Cancer Dataset
184 print("\n=== REAL-WORLD EXAMPLE - BREAST CANCER DATASET ===")
185 cancer = load_breast_cancer()
186 X_cancer, y_cancer = cancer.data, cancer.target
187 X_cancer_train, X_cancer_test, y_cancer_train, y_cancer_test = train_test_split(
188     X_cancer, y_cancer, test_size=0.2, random_state=42)
189
190

```

```

191
192 # Scale the data
193 scaler_cancer = StandardScaler()
194 X_cancer_train_scaled = scaler_cancer.fit_transform(X_cancer_train)
195 X_cancer_test_scaled = scaler_cancer.transform(X_cancer_test)
196
197 # Train SVM
198 svm_cancer = SVC(kernel='rbf', random_state=42)
199 svm_cancer.fit(X_cancer_train_scaled, y_cancer_train)
200 cancer_pred = svm_cancer.predict(X_cancer_test_scaled)
201 cancer_accuracy = accuracy_score(y_cancer_test, cancer_pred)
202
203
204 print(f"Breast Cancer Dataset – SVM Accuracy: {cancer_accuracy:.3f}")
205 print(f"Number of support vectors: {svm_cancer.n_support}")
206
207 # 12. Probability Calibration
208 print("\n=== PROBABILITY CALIBRATION ===")
209 # SVM can output probabilities, but they need to be calibrated
210 svm_prob = SVC(kernel='rbf', probability=True, random_state=42)
211 svm_prob.fit(X4_train_scaled, y4_train)
212 probabilities = svm_prob.predict_proba(X4_test_scaled)[: , 1]
213
214 print(f"Probability range: {probabilities.min():.3f} to {probabilities.max():.3f}")
215 print(f"Mean probability: {probabilities.mean():.3f}")
216
217 # 13. Model Comparison
218 print("\n=== MODEL COMPARISON ===")
219 models = {
220     'Linear SVM': SVC(kernel='linear', random_state=42),
221     'RBF SVM': SVC(kernel='rbf', random_state=42),
222     'Polynomial SVM': SVC(kernel='poly', degree=3, random_state=42),
223     'Sigmoid SVM': SVC(kernel='sigmoid', random_state=42)
224 }
225
226
227 results = []
228 for name, model in models.items():
229     model.fit(X4_train_scaled, y4_train)
230     accuracy = model.score(X4_test_scaled, y4_test)
231     results.append({'Model': name, 'Accuracy': accuracy})
232     print(f"{name}: {accuracy:.3f}")
233
234 results_df = pd.DataFrame(results)
235 print(f"\nResults Summary:")
236 print(results_df)
237
238 # 14. Summary
239 print("\n=== SVM SUMMARY ===")
240 print("1. SVM finds the best boundary to separate classes")
241 print("2. Support vectors are the most important data points")
242 print("3. Different kernels handle different data types")
243 print("4. C parameter controls error tolerance")
244 print("5. Gamma parameter controls boundary complexity (RBF kernel)")
245 print("6. SVM requires feature scaling")
246 print("7. Works well with high-dimensional data")
247 print("8. Memory efficient (only stores support vectors)")
248 print("9. Good generalization performance")
249 print("10. Hyperparameter tuning is important for optimal performance")

```

3. K-Nearest Neighbors (KNN)

What is KNN?

K-Nearest Neighbors is a simple, instance-based learning algorithm that classifies data points based on the majority class of their k nearest neighbors. It's like asking your k closest friends for their opinion and going with the majority.

Key Concepts:

- **Distance Metric:** Usually Euclidean distance, but can be Manhattan, Minkowski, etc.
- **K Value:** Number of neighbors to consider (odd numbers work better for binary classification)
- **Lazy Learning:** No training phase - all computation happens during prediction
- **Feature Scaling:** Very important since KNN is distance-based

How it Works:

1. Calculate distance from test point to all training points
2. Find k nearest neighbors
3. Take majority vote (classification) or average (regression)
4. Return the result

Advantages:

- Simple to understand and implement
- No assumptions about data distribution
- Works well for non-linear problems
- Can be used for both classification and regression

Disadvantages:

- Computationally expensive for large datasets
- Sensitive to irrelevant features
- Requires feature scaling
- Memory intensive (stores all training data)

In [...

```
1
2 # K-Nearest Neighbors (KNN) - Simplified Examples
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from sklearn.datasets import make_classification, make_blobs, load_digits
7 from sklearn.model_selection import train_test_split, cross_val_score
8 from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
9 from sklearn.preprocessing import StandardScaler
10 from sklearn.metrics import classification_report, confusion_matrix
```

```

11 import warnings
12 warnings.filterwarnings('ignore')
13
14 # Create datasets for demonstration
15 print("=== CREATING DATASETS ===")
16
17 # Dataset 1: Classification
18 X1, y1 = make_classification(n_samples=1000, n_features=2, n_redu
19                             n_clusters_per_class=1, random_state=4
20
21 # Dataset 2: Regression
22 X2, y2 = make_blobs(n_samples=1000, centers=4, n_features=2, rand
23 # Convert to regression by using distance from center
24 y2_reg = np.sqrt(np.sum((X2 - X2.mean(axis=0))**2, axis=1))
25
26 # Dataset 3: High-dimensional data
27 X3, y3 = make_classification(n_samples=1000, n_features=20, n_red
28                             n_clusters_per_class=1, random_state=4
29
30
31 print(f"Dataset 1 (Classification): {X1.shape}, classes: {np.uniq
32 print(f"Dataset 2 (Regression): {X2.shape}, target range: {y2_reg
33 print(f"Dataset 3 (High-dim): {X3.shape}, classes: {np.unique(y3)
34
35 # Split datasets
36 X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, t
37 X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2_re
38 X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, t
39
40
41 # 1. Basic KNN Classification
42 print("\n=== BASIC KNN CLASSIFICATION ===")
43 knn = KNeighborsClassifier(n_neighbors=5)
44 knn.fit(X1_train, y1_train)
45 knn_pred = knn.predict(X1_test)
46 knn_accuracy = accuracy_score(y1_test, knn_pred)
47 print(f"KNN Accuracy (k=5): {knn_accuracy:.3f}")
48
49 # 2. Finding Optimal K Value
50 print("\n=== FINDING OPTIMAL K VALUE ===")
51 k_values = range(1, 21, 2) # Odd numbers from 1 to 19
52 k_scores = []
53
54 for k in k_values:
55     knn_temp = KNeighborsClassifier(n_neighbors=k)
56     knn_temp.fit(X1_train, y1_train)
57     score = knn_temp.score(X1_test, y1_test)
58     k_scores.append(score)
59     print(f"k={k}: Accuracy={score:.3f}")
60
61 # Find best k
62 best_k = k_values[np.argmax(k_scores)]
63 print(f"Best k value: {best_k}")
64
65 # Plot k vs accuracy
66 plt.figure(figsize=(10, 6))
67 plt.plot(k_values, k_scores, 'o-')
68 plt.xlabel('k Value')
69 plt.ylabel('Accuracy')
70 plt.title('KNN: k Value vs Accuracy')
71 plt.grid(True)
72 plt.show()
73
74

```

```

75
76 # 3. Different Distance Metrics
77 print("\n=== DIFFERENT DISTANCE METRICS ===")
78 distance_metrics = ['euclidean', 'manhattan', 'minkowski', 'cheby
79 for metric in distance_metrics:
80     knn_temp = KNeighborsClassifier(n_neighbors=5, metric=metric)
81     knn_temp.fit(X1_train, y1_train)
82     score = knn_temp.score(X1_test, y1_test)
83     print(f"{metric.capitalize()} distance: {score:.3f}")
84
85 # 4. KNN Regression
86 print("\n=== KNN REGRESSION ===")
87 knn_reg = KNeighborsRegressor(n_neighbors=5)
88 knn_reg.fit(X2_train, y2_train)
89 y2_pred = knn_reg.predict(X2_test)
90 mse = mean_squared_error(y2_test, y2_pred)
91 r2 = r2_score(y2_test, y2_pred)
92 print(f"KNN Regression MSE: {mse:.3f}")
93 print(f"KNN Regression R2: {r2:.3f}")
94
95 # 5. Impact of Feature Scaling
96 print("\n=== IMPACT OF FEATURE SCALING ===")
97 # Create data with different scales
98 X_scaled = X3.copy()
99 X_scaled[:, 0] = X_scaled[:, 0] * 1000 # Scale first feature
100 X_scaled[:, 1] = X_scaled[:, 1] * 0.001 # Scale second feature
101
102 X_scaled_train, X_scaled_test, y_scaled_train, y_scaled_test = tr
103     X_scaled, y3, test_size=0.2, random_state=42
104 )
105
106 # Without scaling
107 knn_no_scale = KNeighborsClassifier(n_neighbors=5)
108 knn_no_scale.fit(X_scaled_train, y_scaled_train)
109 no_scale_accuracy = knn_no_scale.score(X_scaled_test, y_scaled_te
110 print(f"Without scaling: {no_scale_accuracy:.3f}")
111
112 # With scaling
113 scaler = StandardScaler()
114 X_scaled_train_scaled = scaler.fit_transform(X_scaled_train)
115 X_scaled_test_scaled = scaler.transform(X_scaled_test)
116
117 knn_scaled = KNeighborsClassifier(n_neighbors=5)
118 knn_scaled.fit(X_scaled_train_scaled, y_scaled_train)
119 scaled_accuracy = knn_scaled.score(X_scaled_test_scaled, y_scaled
120 print(f"With scaling: {scaled_accuracy:.3f}")
121
122 # 6. Weighted KNN
123 print("\n=== WEIGHTED KNN ===")
124 # Distance-based weighting: closer neighbors have more influence
125 knn_weighted = KNeighborsClassifier(n_neighbors=5, weights='dista
126 knn_weighted.fit(X1_train, y1_train)
127 weighted_accuracy = knn_weighted.score(X1_test, y1_test)
128 print(f"Weighted KNN Accuracy: {weighted_accuracy:.3f}")
129
130 # 7. Cross-Validation
131 print("\n=== CROSS-VALIDATION ===")
132 cv_scores = cross_val_score(KNeighborsClassifier(n_neighbors=5),
133     print(f"Cross-validation scores: {cv_scores}")
134     print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std

```

```

139
140 # 8. Hyperparameter Tuning
141 print("\n=== HYPERPARAMETER TUNING ===")
142 param_grid = {
143     'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
144     'weights': ['uniform', 'distance'],
145     'metric': ['euclidean', 'manhattan', 'minkowski']
146 }
147
148 grid_search = GridSearchCV(
149     KNeighborsClassifier(),
150     param_grid,
151     cv=5,
152     scoring='accuracy',
153     n_jobs=-1
154 )
155
156 grid_search.fit(X1_train, y1_train)
157 print(f"Best parameters: {grid_search.best_params_}")
158 print(f"Best cross-validation score: {grid_search.best_score_:3f}")
159 print(f"Test accuracy with best params: {grid_search.score(X1_test, y1_test):3f}")
160
161 # 9. Decision Boundary Visualization
162 print("\n=== DECISION BOUNDARY VISUALIZATION ===")
163 def plot_knn_decision_boundary(X, y, k, title):
164     h = 0.02
165     x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
166     y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
167     xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
168                          np.arange(y_min, y_max, h))
169
170     knn_temp = KNeighborsClassifier(n_neighbors=k)
171     knn_temp.fit(X, y)
172     Z = knn_temp.predict(np.c_[xx.ravel(), yy.ravel()])
173     Z = Z.reshape(xx.shape)
174
175     plt.figure(figsize=(8, 6))
176     plt.contourf(xx, yy, Z, alpha=0.8, cmap=plt.cm.RdYlBu)
177     plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdYlBu, edgecolor='k')
178     plt.title(f'{title} (k={k})')
179     plt.xlabel('Feature 1')
180     plt.ylabel('Feature 2')
181     plt.show()
182
183 # Plot decision boundaries for different k values
184 for k in [1, 5, 15]:
185     plot_knn_decision_boundary(X1, y1, k, f'KNN Decision Boundary (k={k})')
186
187 # 10. Real-world Example - Breast Cancer Dataset
188 print("\n=== REAL-WORLD EXAMPLE - BREAST CANCER DATASET ===")
189 cancer = load_breast_cancer()
190 X_cancer, y_cancer = cancer.data, cancer.target
191 X_cancer_train, X_cancer_test, y_cancer_train, y_cancer_test = train_test_split(
192     X_cancer, y_cancer, test_size=0.2, random_state=42)
193
194 # Scale the data
195 scaler_cancer = StandardScaler()
196 X_cancer_train_scaled = scaler_cancer.fit_transform(X_cancer_train)
197 X_cancer_test_scaled = scaler_cancer.transform(X_cancer_test)

```

```

203
204 # Train KNN
205 knn_cancer = KNeighborsClassifier(n_neighbors=5)
206 knn_cancer.fit(X_cancer_train_scaled, y_cancer_train)
207 cancer_pred = knn_cancer.predict(X_cancer_test_scaled)
208 cancer_accuracy = accuracy_score(y_cancer_test, cancer_pred)
209
210 print(f"Breast Cancer Dataset - KNN Accuracy: {cancer_accuracy:.3f}")
211
212 # 11. KNN for Different Problem Sizes
213 print("\n=== KNN FOR DIFFERENT PROBLEM SIZES ===")
214 # Test KNN performance with different dataset sizes
215 sizes = [100, 500, 1000, 2000, 5000]
216 for size in sizes:
217     if size <= len(X1):
218         # Sample data
219         indices = np.random.choice(len(X1), size=size, replace=False)
220         X_sample = X1[indices]
221         y_sample = y1[indices]
222
223         # Split
224         X_sample_train, X_sample_test, y_sample_train, y_sample_test = train_test_split(
225             X_sample, y_sample, test_size=0.2, random_state=42)
226
227         # Train and test
228         knn_temp = KNeighborsClassifier(n_neighbors=5)
229         knn_temp.fit(X_sample_train, y_sample_train)
230         accuracy = knn_temp.score(X_sample_test, y_sample_test)
231         print(f"Dataset size {size}: Accuracy={accuracy:.3f}")
232
233 # 12. KNN vs Other Algorithms
234 print("\n=== KNN VS OTHER ALGORITHMS ===")
235 from sklearn.ensemble import RandomForestClassifier
236 from sklearn.svm import SVC
237 from sklearn.linear_model import LogisticRegression
238
239 # Scale data for fair comparison
240 scaler_comp = StandardScaler()
241 X1_train_scaled = scaler_comp.fit_transform(X1_train)
242 X1_test_scaled = scaler_comp.transform(X1_test)
243
244 models = {
245     'KNN': KNeighborsClassifier(n_neighbors=5),
246     'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
247     'SVM': SVC(kernel='rbf', random_state=42),
248     'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000)
249 }
250
251 results = []
252 for name, model in models.items():
253     model.fit(X1_train_scaled, y1_train)
254     accuracy = model.score(X1_test_scaled, y1_test)
255     results.append({'Model': name, 'Accuracy': accuracy})
256     print(f"{name}: {accuracy:.3f}")
257
258 results_df = pd.DataFrame(results)
259 print(f"\nResults Summary:")
260 print(results_df)
261

```



```

267 # 13. KNN Advantages and Limitations
268 print("\n=== KNN ADVANTAGES AND LIMITATIONS ===")
269 print("Advantages:")
270 print("- Simple to understand and implement")
271 print("- No assumptions about data distribution")
272 print("- Works well for non-linear problems")
273 print("- Can be used for both classification and regression")
274 print("- Good for small to medium datasets")
275
276 print("\nLimitations:")
277 print("- Computationally expensive for large datasets")
278 print("- Sensitive to irrelevant features")
print("- Requires feature scaling")
print("- Memory intensive (stores all training data)")
print("- Sensitive to the curse of dimensionality")

# 14. Summary
print("\n=== KNN SUMMARY ===")
print("1. KNN is a simple, instance-based learning algorithm")
print("2. It classifies based on the majority of k nearest neighbors")
print("3. The choice of k is crucial - odd numbers work better for classification")
print("4. Distance metric affects performance - Euclidean is most common")
print("5. Feature scaling is essential for good performance")
print("6. Weighted KNN gives more influence to closer neighbors")
print("7. Cross-validation helps find optimal parameters")
print("8. Works well for small to medium datasets")
print("9. Can be used for both classification and regression")
print("10. Simple to understand but can be slow for large datasets")

```

4. Clustering Algorithms (K-means, Hierarchical)

What is Clustering?

Clustering is an unsupervised learning technique that groups similar data points together without knowing the true labels. It's like organizing a messy room by putting similar items together.

Key Concepts:

- **Centroid:** The center point of a cluster (for K-means)
- **Distance Metric:** How we measure similarity between data points
- **Cluster Assignment:** Which cluster each data point belongs to
- **Cluster Evaluation:** How well the clustering performed

Types of Clustering:

- **K-means:** Partitions data into k clusters with centroids
- **Hierarchical:** Creates a tree of clusters (dendrogram)
- **DBSCAN:** Density-based clustering, finds arbitrary shaped clusters
- **Gaussian Mixture:** Probabilistic clustering

Applications:

- Customer segmentation
- Image segmentation
- Gene sequencing
- Market research
- Anomaly detection

In [...

```

1
2  # Unsupervised learning: Clustering and dimensionality reduction
3  import pandas as pd
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import make_blobs, make_circles, make_moons
7  from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
8  from sklearn.decomposition import PCA
9  from sklearn.manifold import TSNE
10 from sklearn.preprocessing import StandardScaler
11 from sklearn.metrics import silhouette_score, adjusted_rand_score
12 import warnings
13 warnings.filterwarnings('ignore')
14
15 # Create synthetic datasets for clustering
16 print("=== CREATING SYNTHETIC DATASETS FOR CLUSTERING ===")
17
18 # Dataset 1: Well-separated clusters
19 X1, y1 = make_blobs(n_samples=300, centers=4, n_features=2, random_state=42)
20
21 # Dataset 2: Circular clusters
22 X2, y2 = make_circles(n_samples=300, noise=0.1, factor=0.3, random_state=42)
23
24 # Dataset 3: Moon-shaped clusters
25 X3, y3 = make_moons(n_samples=300, noise=0.1, random_state=42)
26
27 # Dataset 4: High-dimensional data
28 X4, y4 = make_blobs(n_samples=300, centers=3, n_features=10, random_state=42)
29
30 print(f"Dataset 1 shape: {X1.shape}, true clusters: {len(np.unique(y1))}")
31 print(f"Dataset 2 shape: {X2.shape}, true clusters: {len(np.unique(y2))}")
32 print(f"Dataset 3 shape: {X3.shape}, true clusters: {len(np.unique(y3))}")
33 print(f"Dataset 4 shape: {X4.shape}, true clusters: {len(np.unique(y4))}")
34
35 # 1. K-Means Clustering
36 print("\n=== K-MEANS CLUSTERING ===")
37
38 # K-Means on well-separated clusters
39 kmeans1 = KMeans(n_clusters=4, random_state=42)
40 kmeans1.fit(X1)
41 y1_pred_kmeans = kmeans1.predict(X1)
42
43 # K-Means on circular clusters
44 kmeans2 = KMeans(n_clusters=2, random_state=42)
45 kmeans2.fit(X2)
46 y2_pred_kmeans = kmeans2.predict(X2)
47
48 # K-Means on moon-shaped clusters
49 kmeans3 = KMeans(n_clusters=2, random_state=42)
50 kmeans3.fit(X3)
51 y3_pred_kmeans = kmeans3.predict(X3)
52
53
54

```

```

55
56 # Evaluate K-Means
57 silhouette1 = silhouette_score(X1, y1_pred_kmeans)
58 silhouette2 = silhouette_score(X2, y2_pred_kmeans)
59 silhouette3 = silhouette_score(X3, y3_pred_kmeans)
60
61 ari1 = adjusted_rand_score(y1, y1_pred_kmeans)
62 ari2 = adjusted_rand_score(y2, y2_pred_kmeans)
63 ari3 = adjusted_rand_score(y3, y3_pred_kmeans)
64
65 print(f"K-Means on Dataset 1: Silhouette={silhouette1:.3f}, ARI={
66 print(f"K-Means on Dataset 2: Silhouette={silhouette2:.3f}, ARI={
67 print(f"K-Means on Dataset 3: Silhouette={silhouette3:.3f}, ARI={
68
69
70 # 2. DBSCAN Clustering
71 print("\n=== DBSCAN CLUSTERING ===")
72
73 # DBSCAN on well-separated clusters
74 dbscan1 = DBSCAN(eps=0.5, min_samples=5)
75 dbscan1.fit(X1)
76 y1_pred_dbscan = dbscan1.labels_
77
78 # DBSCAN on circular clusters
79 dbscan2 = DBSCAN(eps=0.3, min_samples=5)
80 dbscan2.fit(X2)
81 y2_pred_dbscan = dbscan2.labels_
82
83 # DBSCAN on moon-shaped clusters
84 dbscan3 = DBSCAN(eps=0.3, min_samples=5)
85 dbscan3.fit(X3)
86 y3_pred_dbscan = dbscan3.labels_
87
88
89 # Evaluate DBSCAN
90 silhouette1_db = silhouette_score(X1, y1_pred_dbscan)
91 silhouette2_db = silhouette_score(X2, y2_pred_dbscan)
92 silhouette3_db = silhouette_score(X3, y3_pred_dbscan)
93
94 ari1_db = adjusted_rand_score(y1, y1_pred_dbscan)
95 ari2_db = adjusted_rand_score(y2, y2_pred_dbscan)
96 ari3_db = adjusted_rand_score(y3, y3_pred_dbscan)
97
98 print(f"DBSCAN on Dataset 1: Silhouette={silhouette1_db:.3f}, ARI=
99 print(f"DBSCAN on Dataset 2: Silhouette={silhouette2_db:.3f}, ARI=
100 print(f"DBSCAN on Dataset 3: Silhouette={silhouette3_db:.3f}, ARI=
101
102
103 # 3. Hierarchical Clustering
104 print("\n=== HIERARCHICAL CLUSTERING ===")
105
106 # Hierarchical clustering on well-separated clusters
107 hierarchical1 = AgglomerativeClustering(n_clusters=4)
108 hierarchical1.fit(X1)
109 y1_pred_hier = hierarchical1.labels_
110
111 # Hierarchical clustering on circular clusters
112 hierarchical2 = AgglomerativeClustering(n_clusters=2)
113 hierarchical2.fit(X2)
114 y2_pred_hier = hierarchical2.labels_
115
116 # Hierarchical clustering on moon-shaped clusters
117 hierarchical3 = AgglomerativeClustering(n_clusters=2)
118

```

```

119 hierarchical3.fit(X3)
120 y3_pred_hier = hierarchical3.labels_
121
122 # Evaluate Hierarchical
123 silhouette1_hier = silhouette_score(X1, y1_pred_hier)
124 silhouette2_hier = silhouette_score(X2, y2_pred_hier)
125 silhouette3_hier = silhouette_score(X3, y3_pred_hier)
126
127 ari1_hier = adjusted_rand_score(y1, y1_pred_hier)
128 ari2_hier = adjusted_rand_score(y2, y2_pred_hier)
129 ari3_hier = adjusted_rand_score(y3, y3_pred_hier)
130
131
132 print(f"Hierarchical on Dataset 1: Silhouette={silhouette1_hier:.
133 print(f"Hierarchical on Dataset 2: Silhouette={silhouette2_hier:.
134 print(f"Hierarchical on Dataset 3: Silhouette={silhouette3_hier:.
135
136 # 4. Dimensionality Reduction: PCA
137 print("\n=== PRINCIPAL COMPONENT ANALYSIS (PCA) ===")
138
139 # Apply PCA to high-dimensional data
140 pca = PCA(n_components=2)
141 X4_pca = pca.fit_transform(X4)
142
143 print(f"Original dimensions: {X4.shape}")
144 print(f"PCA dimensions: {X4_pca.shape}")
145 print(f"Explained variance ratio: {pca.explained_variance_ratio_}
146 print(f"Cumulative explained variance: {np.cumsum(pca.explained_v
147
148
149 # Apply PCA to all datasets
150 X1_pca = PCA(n_components=2).fit_transform(X1)
151 X2_pca = PCA(n_components=2).fit_transform(X2)
152 X3_pca = PCA(n_components=2).fit_transform(X3)
153
154 # 5. Dimensionality Reduction: t-SNE
155 print("\n=== t-SNE DIMENSIONALITY REDUCTION ===")
156
157 # Apply t-SNE to high-dimensional data
158 tsne = TSNE(n_components=2, random_state=42)
159 X4_tsne = tsne.fit_transform(X4)
160
161 print(f"t-SNE dimensions: {X4_tsne.shape}")
162
163 # Apply t-SNE to all datasets
164 X1_tsne = TSNE(n_components=2, random_state=42).fit_transform(X1)
165 X2_tsne = TSNE(n_components=2, random_state=42).fit_transform(X2)
166 X3_tsne = TSNE(n_components=2, random_state=42).fit_transform(X3)
167
168
169 # 6. Clustering on Reduced Dimensions
170 print("\n=== CLUSTERING ON REDUCED DIMENSIONS ===")
171
172 # K-Means on PCA-reduced data
173 kmeans_pca = KMeans(n_clusters=3, random_state=42)
174 kmeans_pca.fit(X4_pca)
175 y4_pred_kmeans_pca = kmeans_pca.predict(X4_pca)
176
177 # K-Means on t-SNE-reduced data
178 kmeans_tsne = KMeans(n_clusters=3, random_state=42)
179 kmeans_tsne.fit(X4_tsne)
180 y4_pred_kmeans_tsne = kmeans_tsne.predict(X4_tsne)
181
182

```

```

183 # Evaluate clustering on reduced dimensions
184 silhouette_pca = silhouette_score(X4_pca, y4_pred_kmeans_pca)
185 silhouette_tsne = silhouette_score(X4_tsne, y4_pred_kmeans_tsne)
186
187 ari_pca = adjusted_rand_score(y4, y4_pred_kmeans_pca)
188 ari_tsne = adjusted_rand_score(y4, y4_pred_kmeans_tsne)
189
190 print(f"K-Means on PCA data: Silhouette={silhouette_pca:.3f}, ARI
191 print(f"K-Means on t-SNE data: Silhouette={silhouette_tsne:.3f},
192
193 # 7. Optimal Number of Clusters
194 print("\n=== FINDING OPTIMAL NUMBER OF CLUSTERS ===")
195
196 # Elbow method for K-Means
197 k_range = range(1, 11)
198 inertias = []
199 silhouettes = []
200
201
202 for k in k_range:
203     kmeans = KMeans(n_clusters=k, random_state=42)
204     kmeans.fit(X1)
205     inertias.append(kmeans.inertia_)
206     silhouettes.append(silhouette_score(X1, kmeans.labels_))
207
208 # Find optimal k
209 optimal_k = k_range[np.argmax(silhouettes)]
210 print(f"Optimal number of clusters (silhouette): {optimal_k}")
211
212 # 8. Clustering Visualization
213 print("\n=== CLUSTERING VISUALIZATION ===")
214
215 # Create subplots for visualization
216 fig, axes = plt.subplots(3, 4, figsize=(20, 15))
217
218 # Original data
219 axes[0, 0].scatter(X1[:, 0], X1[:, 1], c=y1, cmap='viridis')
220 axes[0, 0].set_title('Original Data (Well-separated)')
221 axes[0, 0].set_xlabel('Feature 1')
222 axes[0, 0].set_ylabel('Feature 2')
223
224 axes[0, 1].scatter(X2[:, 0], X2[:, 1], c=y2, cmap='viridis')
225 axes[0, 1].set_title('Original Data (Circular)')
226 axes[0, 1].set_xlabel('Feature 1')
227 axes[0, 1].set_ylabel('Feature 2')
228
229 axes[0, 2].scatter(X3[:, 0], X3[:, 1], c=y3, cmap='viridis')
230 axes[0, 2].set_title('Original Data (Moon-shaped)')
231 axes[0, 2].set_xlabel('Feature 1')
232 axes[0, 2].set_ylabel('Feature 2')
233
234 axes[0, 3].scatter(X4_pca[:, 0], X4_pca[:, 1], c=y4, cmap='viridi
235 axes[0, 3].set_title('PCA Reduced Data')
236 axes[0, 3].set_xlabel('PC1')
237 axes[0, 3].set_ylabel('PC2')
238
239 # K-Means results
240 axes[1, 0].scatter(X1[:, 0], X1[:, 1], c=y1_pred_kmeans, cmap='vi
241 axes[1, 0].set_title('K-Means (Well-separated)')
242 axes[1, 0].set_xlabel('Feature 1')
243 axes[1, 0].set_ylabel('Feature 2')
244

```

```

247 axes[1, 1].scatter(X2[:, 0], X2[:, 1], c=y2_pred_kmeans, cmap='viridis')
248 axes[1, 1].set_title('K-Means (Circular)')
249 axes[1, 1].set_xlabel('Feature 1')
250 axes[1, 1].set_ylabel('Feature 2')
251
252 axes[1, 2].scatter(X3[:, 0], X3[:, 1], c=y3_pred_kmeans, cmap='viridis')
253 axes[1, 2].set_title('K-Means (Moon-shaped)')
254 axes[1, 2].set_xlabel('Feature 1')
255 axes[1, 2].set_ylabel('Feature 2')
256
257 axes[1, 3].scatter(X4_pca[:, 0], X4_pca[:, 1], c=y4_pred_kmeans_pca)
258 axes[1, 3].set_title('K-Means on PCA Data')
259 axes[1, 3].set_xlabel('PC1')
260 axes[1, 3].set_ylabel('PC2')
261
262 # DBSCAN results
263 axes[2, 0].scatter(X1[:, 0], X1[:, 1], c=y1_pred_dbscan, cmap='viridis')
264 axes[2, 0].set_title('DBSCAN (Well-separated)')
265 axes[2, 0].set_xlabel('Feature 1')
266 axes[2, 0].set_ylabel('Feature 2')
267
268 axes[2, 1].scatter(X2[:, 0], X2[:, 1], c=y2_pred_dbscan, cmap='viridis')
269 axes[2, 1].set_title('DBSCAN (Circular)')
270 axes[2, 1].set_xlabel('Feature 1')
271 axes[2, 1].set_ylabel('Feature 2')
272
273 axes[2, 2].scatter(X3[:, 0], X3[:, 1], c=y3_pred_dbscan, cmap='viridis')
274 axes[2, 2].set_title('DBSCAN (Moon-shaped)')
275 axes[2, 2].set_xlabel('Feature 1')
276 axes[2, 2].set_ylabel('Feature 2')
277
278 axes[2, 3].scatter(X4_tsne[:, 0], X4_tsne[:, 1], c=y4_pred_kmeans)
279 axes[2, 3].set_title('K-Means on t-SNE Data')
280 axes[2, 3].set_xlabel('t-SNE 1')
281 axes[2, 3].set_ylabel('t-SNE 2')
282
283 plt.tight_layout()
284 plt.show()
285
286 # 9. Clustering Summary
287 print("\n=== CLUSTERING SUMMARY ===")
288 print("1. K-Means: Good for spherical clusters, requires number of clusters")
289 print("2. DBSCAN: Good for arbitrary shapes, finds noise points")
290 print("3. Hierarchical: Good for hierarchical structure, can be sensitive to distance metric")
291 print("4. PCA: Linear dimensionality reduction, preserves variance")
292 print("5. t-SNE: Non-linear dimensionality reduction, good for visualization")
293 print("6. Silhouette Score: Measures cluster quality (-1 to 1)")
294 print("7. ARI: Measures clustering accuracy against true labels")
295 print("8. Elbow Method: Helps find optimal number of clusters")
296 print("9. Visualization: Essential for understanding clustering results")
297 print("10. Preprocessing: Scaling often necessary for clustering")

```

5. Dimensionality Reduction (PCA)

What is PCA?

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms data into a lower-dimensional space while preserving as much variance as possible. It's like finding the best angle to view a 3D object to see its most important features.

Key Concepts:

- **Principal Components:** New axes that capture the most variance in the data
- **Eigenvalues:** Measure of variance explained by each component
- **Eigenvectors:** Directions of maximum variance
- **Explained Variance Ratio:** Percentage of total variance explained by each component

How PCA Works:

1. Center the data (subtract mean)
2. Calculate covariance matrix
3. Find eigenvalues and eigenvectors
4. Sort components by eigenvalues (variance)
5. Project data onto top k components

Applications:

- Data visualization (reduce to 2D/3D)
- Noise reduction
- Feature extraction
- Data compression
- Preprocessing for other algorithms

Advantages:

- Reduces overfitting
- Speeds up algorithms
- Removes noise
- Visualizes high-dimensional data
- Preserves most important information

In [...

```
1
2  # Dimensionality Reduction (PCA) - Simplified Examples
3  import pandas as pd
4  import numpy as np
5  import matplotlib.pyplot as plt
6  from sklearn.datasets import make_classification, load_breast_cancer
7  from sklearn.decomposition import PCA
8  from sklearn.manifold import TSNE
9  from sklearn.preprocessing import StandardScaler
10 from sklearn.model_selection import train_test_split
11 from sklearn.ensemble import RandomForestClassifier
12 from sklearn.metrics import accuracy_score
```

```

13 import warnings
14 warnings.filterwarnings('ignore')
15
16 # Create datasets for dimensionality reduction
17 print("=== CREATING DATASETS FOR DIMENSIONALITY REDUCTION ===")
18
19 # Dataset 1: High-dimensional classification data
20 X1, y1 = make_classification(n_samples=1000, n_features=20, n_red
21                             n_clusters_per_class=1, random_state=4
22
23
24 # Dataset 2: Real-world dataset - Breast Cancer
25 cancer = load_breast_cancer()
26 X2, y2 = cancer.data, cancer.target
27
28 # Dataset 3: Real-world dataset - Wine
29 wine = load_wine()
30 X3, y3 = wine.data, wine.target
31
32 print(f"Dataset 1 (Synthetic): {X1.shape}, classes: {len(np.unique(y1))}")
33 print(f"Dataset 2 (Cancer): {X2.shape}, classes: {len(np.unique(y2))}")
34 print(f"Dataset 3 (Wine): {X3.shape}, classes: {len(np.unique(y3))}")
35
36 # 1. Basic PCA - Understanding Components
37 print("\n=== BASIC PCA - UNDERSTANDING COMPONENTS ===")
38
39 # Apply PCA to synthetic data
40 pca1 = PCA()
41 X1_pca = pca1.fit_transform(X1)
42
43
44 print(f"Original dimensions: {X1.shape[1]}")
45 print(f"PCA dimensions: {X1_pca.shape[1]}")
46 print(f"Explained variance ratio: {pca1.explained_variance_ratio_}")
47 print(f"Cumulative explained variance: {np.cumsum(pca1.explained_
48
49 # 2. Choosing Number of Components
50 print("\n=== CHOOSING NUMBER OF COMPONENTS ===")
51
52 # Method 1: Keep components that explain 95% of variance
53 pca_95 = PCA(n_components=0.95)
54 X1_pca_95 = pca_95.fit_transform(X1)
55 print(f"Components for 95% variance: {X1_pca_95.shape[1]}")
56
57 # Method 2: Keep specific number of components
58 pca_5 = PCA(n_components=5)
59 X1_pca_5 = pca_5.fit_transform(X1)
60 print(f"Components for 5 components: {X1_pca_5.shape[1]}")
61 print(f"Explained variance with 5 components: {np.sum(pca_5.explained_
62
63
64 # 3. PCA Visualization - Scree Plot
65 print("\n=== PCA VISUALIZATION - SCREE PLOT ===")
66
67 # Plot explained variance ratio
68 plt.figure(figsize=(12, 5))
69
70 plt.subplot(1, 2, 1)
71 plt.plot(range(1, len(pca1.explained_variance_ratio_) + 1), pca1.
72 plt.xlabel('Principal Component')
73 plt.ylabel('Explained Variance Ratio')
74 plt.title('Explained Variance by Component')
75 plt.grid(True)
76

```



```

77
78 plt.subplot(1, 2, 2)
79 plt.plot(range(1, len(pca1.explained_variance_ratio_) + 1), np.cu
80 plt.xlabel('Number of Components')
81 plt.ylabel('Cumulative Explained Variance')
82 plt.title('Cumulative Explained Variance')
83 plt.axhline(y=0.95, color='r', linestyle='--', label='95% Variance
84 plt.legend()
85 plt.grid(True)
86
87
88 plt.tight_layout()
89 plt.show()
90
91 # 4. PCA for Data Visualization
92 print("\n=== PCA FOR DATA VISUALIZATION ===")
93
94 # Reduce to 2D for visualization
95 pca_2d = PCA(n_components=2)
96 X1_pca_2d = pca_2d.fit_transform(X1)
97
98 # Plot original data (first 2 features)
99 plt.figure(figsize=(12, 5))
100
101 plt.subplot(1, 2, 1)
102 plt.scatter(X1[:, 0], X1[:, 1], c=y1, cmap='viridis', alpha=0.7)
103 plt.xlabel('Feature 1')
104 plt.ylabel('Feature 2')
105 plt.title('Original Data (First 2 Features)')
106 plt.colorbar()
107
108 plt.subplot(1, 2, 2)
109 plt.scatter(X1_pca_2d[:, 0], X1_pca_2d[:, 1], c=y1, cmap='viridis
110 plt.xlabel('First Principal Component')
111 plt.ylabel('Second Principal Component')
112 plt.title('PCA Reduced Data (2D)')
113 plt.colorbar()
114
115
116 plt.tight_layout()
117 plt.show()
118
119 # 5. PCA on Real-world Data
120 print("\n=== PCA ON REAL-WORLD DATA ===")
121
122 # Apply PCA to breast cancer dataset
123 pca_cancer = PCA()
124 X2_pca = pca_cancer.fit_transform(X2)
125
126 print(f"Cancer dataset - Original: {X2.shape}")
127 print(f"Cancer dataset - PCA: {X2_pca.shape}")
128 print(f"First 5 components explain {np.sum(pca_cancer.explained_v
129
130
131 # Reduce to 2D for visualization
132 pca_cancer_2d = PCA(n_components=2)
133 X2_pca_2d = pca_cancer_2d.fit_transform(X2)
134
135 plt.figure(figsize=(8, 6))
136 plt.scatter(X2_pca_2d[:, 0], X2_pca_2d[:, 1], c=y2, cmap='viridis
137 plt.xlabel('First Principal Component')
138 plt.ylabel('Second Principal Component')
139 plt.title('Breast Cancer Dataset - PCA Visualization')
140

```



```

141 plt.colorbar()
142 plt.show()
143
144 # 6. PCA for Feature Selection
145 print("\n=== PCA FOR FEATURE SELECTION ===")
146
147 # Compare classification performance with and without PCA
148 X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, t
149
150 # Without PCA
151 rf_original = RandomForestClassifier(n_estimators=100, random_sta
152 rf_original.fit(X1_train, y1_train)
153 y1_pred_original = rf_original.predict(X1_test)
154 accuracy_original = accuracy_score(y1_test, y1_pred_original)
155
156 # With PCA (keeping 95% variance)
157 X1_train_pca = pca_95.fit_transform(X1_train)
158 X1_test_pca = pca_95.transform(X1_test)
159
160 rf_pca = RandomForestClassifier(n_estimators=100, random_state=42
161 rf_pca.fit(X1_train_pca, y1_train)
162 y1_pred_pca = rf_pca.predict(X1_test_pca)
163 accuracy_pca = accuracy_score(y1_test, y1_pred_pca)
164
165 print(f"Accuracy without PCA: {accuracy_original:.3f}")
166 print(f"Accuracy with PCA (95% variance): {accuracy_pca:.3f}")
167 print(f"Features reduced from {X1_train.shape[1]} to {X1_train_pc
168
169 # 7. PCA vs t-SNE Comparison
170 print("\n=== PCA VS t-SNE COMPARISON ===")
171
172 # Apply both PCA and t-SNE to wine dataset
173 pca_wine = PCA(n_components=2)
174 X3_pca = pca_wine.fit_transform(X3)
175
176 tsne_wine = TSNE(n_components=2, random_state=42)
177 X3_tsne = tsne_wine.fit_transform(X3)
178
179 # Plot comparison
180 plt.figure(figsize=(12, 5))
181
182 plt.subplot(1, 2, 1)
183 plt.scatter(X3_pca[:, 0], X3_pca[:, 1], c=y3, cmap='viridis', alp
184 plt.xlabel('First Principal Component')
185 plt.ylabel('Second Principal Component')
186 plt.title('PCA on Wine Dataset')
187 plt.colorbar()
188
189 plt.subplot(1, 2, 2)
190 plt.scatter(X3_tsne[:, 0], X3_tsne[:, 1], c=y3, cmap='viridis', a
191 plt.xlabel('t-SNE 1')
192 plt.ylabel('t-SNE 2')
193 plt.title('t-SNE on Wine Dataset')
194 plt.colorbar()
195
196 plt.tight_layout()
197 plt.show()
198
199 # 8. PCA for Noise Reduction
200 print("\n=== PCA FOR NOISE REDUCTION ===")
201
202
203
204

```

```

205
206 # Add noise to data
207 noise = np.random.normal(0, 0.1, X1.shape)
208 X1_noisy = X1 + noise
209
210 # Apply PCA and reconstruct
211 pca_noise = PCA(n_components=10) # Keep 10 components
212 X1_noisy_pca = pca_noise.fit_transform(X1_noisy)
213 X1_reconstructed = pca_noise.inverse_transform(X1_noisy_pca)
214
215 # Calculate reconstruction error
216 mse = np.mean((X1 - X1_reconstructed) ** 2)
217 print(f"Reconstruction MSE: {mse:.4f}")
218
219 # Visualize noise reduction
220 plt.figure(figsize=(12, 5))
221
222
223 plt.subplot(1, 2, 1)
224 plt.scatter(X1_noisy[:, 0], X1_noisy[:, 1], c=y1, cmap='viridis',
225 plt.title('Noisy Data')
226 plt.xlabel('Feature 1')
227 plt.ylabel('Feature 2')
228
229 plt.subplot(1, 2, 2)
230 plt.scatter(X1_reconstructed[:, 0], X1_reconstructed[:, 1], c=y1,
231 plt.title('PCA Reconstructed Data')
232 plt.xlabel('Feature 1')
233 plt.ylabel('Feature 2')
234
235
236 plt.tight_layout()
237 plt.show()
238
239 # 9. PCA for Different Datasets
240 print("\n=== PCA FOR DIFFERENT DATASETS ===")
241
242 datasets = {
243     'Synthetic': (X1, y1),
244     'Cancer': (X2, y2),
245     'Wine': (X3, y3)
246 }
247
248 for name, (X, y) in datasets.items():
249     pca_temp = PCA()
250     X_pca = pca_temp.fit_transform(X)
251
252     # Find number of components for 95% variance
253     cumsum = np.cumsum(pca_temp.explained_variance_ratio_)
254     n_components_95 = np.argmax(cumsum >= 0.95) + 1
255
256     print(f"{name} dataset:")
257     print(f"    Original dimensions: {X.shape[1]}")
258     print(f"    Components for 95% variance: {n_components_95}")
259     print(f"    Variance explained by first 2 components: {np.sum(p
260
261
262 # 10. PCA Biplot (Component Loadings)
263 print("\n=== PCA BILOT (COMPONENT LOADINGS) ===")
264
265 # Create biplot for wine dataset
266 pca_biplot = PCA(n_components=2)
267 X3_pca_biplot = pca_biplot.fit_transform(X3)
268

```

```

269
270 # Get feature names
271 feature_names = wine.feature_names
272
273 # Plot biplot
274 plt.figure(figsize=(10, 8))
275 plt.scatter(X3_pca_biplot[:, 0], X3_pca_biplot[:, 1], c=y3, cmap=
276
277 # Plot feature vectors
278 for i, feature in enumerate(feature_names):
279     plt.arrow(0, 0, pca_biplot.components_[0, i] * 3, pca_biplot.
280             head_width=0.05, head_length=0.05, fc='red', ec='re
281     plt.text(pca_biplot.components_[0, i] * 3.2, pca_biplot.compo
282             fontsize=8, ha='center', va='center')
283
284
285 plt.xlabel('First Principal Component')
286 plt.ylabel('Second Principal Component')
287 plt.title('PCA Biplot - Wine Dataset')
288 plt.grid(True)
289 plt.show()
290
291 # 11. PCA Limitations and Considerations
292 print("\n=== PCA LIMITATIONS AND CONSIDERATIONS ===")
293
294 print("PCA Limitations:")
295 print("1. Linear transformation only - cannot capture non-linear
296 print("2. Assumes data is centered and scaled")
297 print("3. Sensitive to outliers")
298 print("4. May not preserve class separability")
299 print("5. Components are linear combinations of original features
300
301
302 print("\nWhen to use PCA:")
303 print("1. High-dimensional data visualization")
304 print("2. Noise reduction")
305 print("3. Data compression")
306 print("4. Preprocessing for linear algorithms")
307 print("5. Feature extraction")
308
309
310 print("\nWhen NOT to use PCA:")
311 print("1. Non-linear relationships in data")
312 print("2. When interpretability is important")
313 print("3. When all features are equally important")
314 print("4. For non-linear algorithms (may not help)")
315
316 # 12. Summary
317 print("\n=== PCA SUMMARY ===")
318 print("1. PCA reduces dimensionality while preserving variance")
319 print("2. Principal components are orthogonal directions of maxim
320 print("3. Explained variance ratio shows importance of each compo
321 print("4. Choose number of components based on variance threshold
322 print("5. PCA is useful for visualization and noise reduction")
323 print("6. Always scale data before applying PCA")
324 print("7. PCA is linear - use t-SNE for non-linear relationships")
325 print("8. Biplots show both data points and feature contributions")
326 print("9. PCA can improve performance of some algorithms")
327 print("10. Consider trade-off between dimensionality and informat

```

6. Model Hyperparameter Tuning

What is Hyperparameter Tuning?

Hyperparameter tuning is the process of finding the best combination of hyperparameters (parameters that are set before training) for a machine learning model. It's like adjusting the knobs on a machine to get the best performance.

Key Concepts:

- **Hyperparameters:** Parameters set before training (e.g., learning rate, number of trees)
- **Parameters:** Learned during training (e.g., weights, biases)
- **Grid Search:** Exhaustively searches through a specified parameter grid
- **Random Search:** Randomly samples from parameter distributions
- **Cross-Validation:** Evaluates model performance on different data splits

Common Hyperparameters:

- **Random Forest:** `n_estimators`, `max_depth`, `min_samples_split`
- **SVM:** `C`, `gamma`, `kernel`
- **KNN:** `n_neighbors`, `weights`, `metric`
- **Neural Networks:** `learning_rate`, `hidden_layers`, `batch_size`

Tuning Strategies:

1. **Grid Search:** Try all combinations in a grid
2. **Random Search:** Randomly sample parameter combinations
3. **Bayesian Optimization:** Use previous results to guide search
4. **Manual Search:** Use domain knowledge and experience

Best Practices:

- Use cross-validation for robust evaluation
- Start with coarse grid, then refine
- Consider computational cost
- Use early stopping when possible
- Validate on holdout set

In [...

```
1      # Model Hyperparameter Tuning - Simplified Examples
2      import pandas as pd
3      import numpy as np
4      import matplotlib.pyplot as plt
5      from sklearn.datasets import make_classification, load_breast_cancer
6      from sklearn.model_selection import train_test_split, GridSearchCV
7      from sklearn.ensemble import RandomForestClassifier
8      from sklearn.svm import SVC
9      from sklearn.neighbors import KNeighborsClassifier
10     from sklearn.linear_model import LogisticRegression
```

```

13 from sklearn.metrics import accuracy_score, classification_report
14 from sklearn.preprocessing import StandardScaler
15 import time
16 import warnings
17 warnings.filterwarnings('ignore')
18
19 # Create datasets for hyperparameter tuning
20 print("=== CREATING DATASETS FOR HYPERPARAMETER TUNING ===")
21
22 # Dataset 1: Synthetic classification data
23 X1, y1 = make_classification(n_samples=1000, n_features=10, n_redundant=5,
24                             n_clusters_per_class=1, random_state=42)
25
26 # Dataset 2: Real-world dataset
27 cancer = load_breast_cancer()
28 X2, y2 = cancer.data, cancer.target
29
30 print(f"Dataset 1 (Synthetic): {X1.shape}, classes: {len(np.unique(y1))}")
31 print(f"Dataset 2 (Cancer): {X2.shape}, classes: {len(np.unique(y2))}")
32
33 # Split datasets
34 X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, test_size=0.2,
35                                                         random_state=42)
36 X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2,
37                                                         random_state=42)
38
39 # Scale the cancer dataset
40 scaler = StandardScaler()
41 X2_train_scaled = scaler.fit_transform(X2_train)
42 X2_test_scaled = scaler.transform(X2_test)
43
44 # 1. Manual Hyperparameter Tuning
45 print("\n=== MANUAL HYPERPARAMETER TUNING ===")
46
47 # Test different values manually
48 n_estimators_list = [50, 100, 200, 500]
49 max_depth_list = [None, 10, 20, 30]
50
51 print("Manual tuning for Random Forest:")
52 best_score = 0
53 best_params = {}
54
55 for n_est in n_estimators_list:
56     for max_d in max_depth_list:
57         rf = RandomForestClassifier(n_estimators=n_est, max_depth=max_d,
58                                   min_samples_split=10, min_samples_leaf=5,
59                                   n_jobs=-1, random_state=42)
60         scores = cross_val_score(rf, X1_train, y1_train, cv=5)
61         mean_score = scores.mean()
62
63         print(f"n_estimators={n_est}, max_depth={max_d}: CV Score {mean_score:.3f}")
64
65         if mean_score > best_score:
66             best_score = mean_score
67             best_params = {'n_estimators': n_est, 'max_depth': max_d}
68
69 print(f"\nBest manual parameters: {best_params}")
70 print(f"Best manual CV score: {best_score:.3f}")
71
72 # 2. Grid Search
73 print("\n=== GRID SEARCH ===")
74
75 # Define parameter grid
76 param_grid = {

```

```

77         'n_estimators': [50, 100, 200],
78         'max_depth': [None, 10, 20],
79         'min_samples_split': [2, 5, 10],
80         'min_samples_leaf': [1, 2, 4]
81     }
82
83     # Create GridSearchCV
84     grid_search = GridSearchCV(
85         RandomForestClassifier(random_state=42),
86         param_grid,
87         cv=5,
88         scoring='accuracy',
89         n_jobs=-1,
90         verbose=1
91     )
92
93
94     # Fit grid search
95     start_time = time.time()
96     grid_search.fit(X1_train, y1_train)
97     grid_time = time.time() - start_time
98
99     print(f"Grid Search completed in {grid_time:.2f} seconds")
100    print(f"Best parameters: {grid_search.best_params_}")
101    print(f"Best cross-validation score: {grid_search.best_score_:.3f}")
102
103    # Test best model
104    best_rf = grid_search.best_estimator_
105    y1_pred_grid = best_rf.predict(X1_test)
106    grid_accuracy = accuracy_score(y1_test, y1_pred_grid)
107    print(f"Test accuracy: {grid_accuracy:.3f}")
108
109
110    # 3. Random Search
111    print("\n=== RANDOM SEARCH ===")
112
113    # Define parameter distributions
114    param_distributions = {
115        'n_estimators': [50, 100, 200, 300, 500],
116        'max_depth': [None, 10, 20, 30, 40],
117        'min_samples_split': [2, 5, 10, 15, 20],
118        'min_samples_leaf': [1, 2, 4, 8, 16],
119        'max_features': ['sqrt', 'log2', None]
120    }
121
122
123    # Create RandomizedSearchCV
124    random_search = RandomizedSearchCV(
125        RandomForestClassifier(random_state=42),
126        param_distributions,
127        n_iter=50, # Number of parameter settings sampled
128        cv=5,
129        scoring='accuracy',
130        n_jobs=-1,
131        random_state=42,
132        verbose=1
133    )
134
135
136    # Fit random search
137    start_time = time.time()
138    random_search.fit(X1_train, y1_train)
139    random_time = time.time() - start_time
140

```

```

141 print(f"Random Search completed in {random_time:.2f} seconds")
142 print(f"Best parameters: {random_search.best_params_}")
143 print(f"Best cross-validation score: {random_search.best_score:.3f}")
144
145 # Test best model
146 best_rf_random = random_search.best_estimator_
147 y1_pred_random = best_rf_random.predict(X1_test)
148 random_accuracy = accuracy_score(y1_test, y1_pred_random)
149 print(f"Test accuracy: {random_accuracy:.3f}")
150
151 # 4. Comparison: Grid Search vs Random Search
152 print("\n=== GRID SEARCH VS RANDOM SEARCH COMPARISON ===")
153
154 print(f"Grid Search:")
155 print(f"  Time: {grid_time:.2f} seconds")
156 print(f"  CV Score: {grid_search.best_score:.3f}")
157 print(f"  Test Accuracy: {grid_accuracy:.3f}")
158
159 print(f"\nRandom Search:")
160 print(f"  Time: {random_time:.2f} seconds")
161 print(f"  CV Score: {random_search.best_score:.3f}")
162 print(f"  Test Accuracy: {random_accuracy:.3f}")
163
164 # 5. Hyperparameter Tuning for Different Algorithms
165 print("\n=== HYPERPARAMETER TUNING FOR DIFFERENT ALGORITHMS ===")
166
167 # Random Forest
168 print("Random Forest tuning:")
169 rf_params = {
170     'n_estimators': [50, 100, 200],
171     'max_depth': [None, 10, 20],
172     'min_samples_split': [2, 5, 10]
173 }
174
175 rf_grid = GridSearchCV(RandomForestClassifier(random_state=42), rf_params, cv=5, n_jobs=-1)
176 rf_grid.fit(X1_train, y1_train)
177 print(f"  Best RF parameters: {rf_grid.best_params_}")
178 print(f"  Best RF CV score: {rf_grid.best_score:.3f}")
179
180 # SVM
181 print("\nSVM tuning:")
182 svm_params = {
183     'C': [0.1, 1, 10, 100],
184     'gamma': [0.001, 0.01, 0.1, 1],
185     'kernel': ['rbf', 'linear', 'poly']
186 }
187
188 svm_grid = GridSearchCV(SVC(random_state=42), svm_params, cv=5, n_jobs=-1)
189 svm_grid.fit(X2_train_scaled, y2_train)
190 print(f"  Best SVM parameters: {svm_grid.best_params_}")
191 print(f"  Best SVM CV score: {svm_grid.best_score:.3f}")
192
193 # KNN
194 print("\nKNN tuning:")
195 knn_params = {
196     'n_neighbors': [3, 5, 7, 9, 11, 15, 21],
197     'weights': ['uniform', 'distance'],
198     'metric': ['euclidean', 'manhattan', 'minkowski']
199 }
200
201 knn_grid = GridSearchCV(KNeighborsClassifier(random_state=42), knn_params, cv=5, n_jobs=-1)
202 knn_grid.fit(X1_train, y1_train)
203 print(f"  Best KNN parameters: {knn_grid.best_params_}")
204 print(f"  Best KNN CV score: {knn_grid.best_score:.3f}")

```



```

205 knn_grid = GridSearchCV(KNeighborsClassifier(), knn_params, cv=5,
206 knn_grid.fit(X2_train_scaled, y2_train)
207 print(f" Best KNN parameters: {knn_grid.best_params_}")
208 print(f" Best KNN CV score: {knn_grid.best_score_:.3f}")
209
210 # 6. Learning Curves for Hyperparameter Tuning
211 print("\n=== LEARNING CURVES FOR HYPERPARAMETER TUNING ===")
212
213 # Test different n_estimators for Random Forest
214 n_estimators_range = [10, 50, 100, 200, 500, 1000]
215 train_scores = []
216 val_scores = []
217
218
219 for n_est in n_estimators_range:
220     rf = RandomForestClassifier(n_estimators=n_est, random_state=
221
222     # Training score
223     rf.fit(X1_train, y1_train)
224     train_score = rf.score(X1_train, y1_train)
225     train_scores.append(train_score)
226
227     # Validation score
228     val_scores_cv = cross_val_score(rf, X1_train, y1_train, cv=5)
229     val_score = val_scores_cv.mean()
230     val_scores.append(val_score)
231
232
233 # Plot learning curves
234 plt.figure(figsize=(10, 6))
235 plt.plot(n_estimators_range, train_scores, 'o-', label='Training
236 plt.plot(n_estimators_range, val_scores, 'o-', label='Validation
237 plt.xlabel('Number of Estimators')
238 plt.ylabel('Accuracy')
239 plt.title('Random Forest Learning Curves')
240 plt.legend()
241 plt.grid(True)
242 plt.show()
243
244 # 7. Hyperparameter Importance Analysis
245 print("\n=== HYPERPARAMETER IMPORTANCE ANALYSIS ===")
246
247 # Analyze results from grid search
248 results_df = pd.DataFrame(grid_search.cv_results_)
249 print("Top 10 parameter combinations:")
250 top_results = results_df.nlargest(10, 'mean_test_score')[['params
251 print(top_results)
252
253
254 # 8. Nested Cross-Validation
255 print("\n=== NESTED CROSS-VALIDATION ===")
256
257 # Outer loop for model evaluation
258 outer_scores = []
259 for i in range(5):
260     # Create train/test split
261     X_train_outer, X_test_outer, y_train_outer, y_test_outer = tr
262     X1, y1, test_size=0.2, random_state=i
263 )
264
265     # Inner loop for hyperparameter tuning
266     inner_grid = GridSearchCV(
267         RandomForestClassifier(random_state=42),
268

```

```

269         param_grid,
270         cv=3,
271         scoring='accuracy'
272     )
273     inner_grid.fit(X_train_outer, y_train_outer)
274
275     # Evaluate best model
276     best_model = inner_grid.best_estimator_
277     score = best_model.score(X_test_outer, y_test_outer)
278     outer_scores.append(score)
279     print(f"Fold {i+1}: Best params = {inner_grid.best_params_},
280
281
282     print(f"\nNested CV scores: {outer_scores}")
283     print(f"Mean nested CV score: {np.mean(outer_scores):.3f} (+/- {n
284
285     # 9. Hyperparameter Tuning Best Practices
286     print("\n=== HYPERPARAMETER TUNING BEST PRACTICES ===")
287
288     print("1. Start with default parameters")
289     print("2. Use cross-validation for robust evaluation")
290     print("3. Start with coarse grid, then refine")
291     print("4. Consider computational cost")
292     print("5. Use early stopping when possible")
293     print("6. Validate on holdout set")
294     print("7. Consider using random search for large parameter spaces")
295     print("8. Use domain knowledge to guide search")
296     print("9. Monitor overfitting")
297     print("10. Document all experiments")
298
299
300     # 10. Real-world Example - Complete Pipeline
301     print("\n=== REAL-WORLD EXAMPLE - COMPLETE PIPELINE ===")
302
303     # Complete pipeline with hyperparameter tuning
304     from sklearn.pipeline import Pipeline
305     from sklearn.preprocessing import StandardScaler
306
307     # Create pipeline
308     pipeline = Pipeline([
309         ('scaler', StandardScaler()),
310         ('classifier', SVC(random_state=42))
311     ])
312
313
314     # Define parameter grid for pipeline
315     pipeline_params = {
316         'classifier__C': [0.1, 1, 10, 100],
317         'classifier__gamma': [0.001, 0.01, 0.1, 1],
318         'classifier__kernel': ['rbf', 'linear']
319     }
320
321
322     # Grid search on pipeline
323     pipeline_grid = GridSearchCV(
324         pipeline,
325         pipeline_params,
326         cv=5,
327         scoring='accuracy',
328         n_jobs=-1
329     )
330
331     # Fit on cancer dataset
332     pipeline_grid.fit(X2_train, y2_train)

```

```

print(f"Best pipeline parameters: {pipeline_grid.best_params_}")
print(f"Best pipeline CV score: {pipeline_grid.best_score_:.3f}")

# Test pipeline
y2_pred_pipeline = pipeline_grid.predict(X2_test)
pipeline_accuracy = accuracy_score(y2_test, y2_pred_pipeline)
print(f"Pipeline test accuracy: {pipeline_accuracy:.3f}")

# 11. Summary
print("\n=== HYPERPARAMETER TUNING SUMMARY ===")
print("1. Hyperparameters are set before training")
print("2. Grid search tries all combinations")
print("3. Random search samples randomly")
print("4. Cross-validation provides robust evaluation")
print("5. Start with coarse grid, then refine")
print("6. Consider computational cost")
print("7. Use early stopping when possible")
print("8. Validate on holdout set")
print("9. Document all experiments")
print("10. Consider using automated tools for complex searches")

```

7. Cross-Validation Strategies

What is Cross-Validation?

Cross-validation is a technique to evaluate how well a model will generalize to new, unseen data. It's like testing a student with multiple different exams to get a fair assessment of their knowledge.

Key Concepts:

- **Training Set:** Data used to train the model
- **Validation Set:** Data used to tune hyperparameters
- **Test Set:** Data used for final evaluation
- **Fold:** One iteration of the cross-validation process
- **Stratified:** Maintains class distribution in each fold

Types of Cross-Validation:

- **K-Fold:** Divides data into k equal parts
- **Stratified K-Fold:** Maintains class proportions
- **Leave-One-Out:** Uses all but one sample for training
- **Time Series Split:** For time-dependent data
- **Shuffle Split:** Random train/test splits

Why Use Cross-Validation?

- **Robust Evaluation:** More reliable than single train/test split
- **Better Use of Data:** Uses all data for both training and validation

- **Detects Overfitting:** Shows if model generalizes well
- **Model Selection:** Compare different algorithms fairly
- **Hyperparameter Tuning:** Find best parameters reliably

Best Practices:

- Use stratified CV for classification
- Use time series CV for temporal data
- Use appropriate number of folds (5-10)
- Ensure data is shuffled before splitting
- Use same CV strategy for all models

In [...

[illegible]

```

47 rf = RandomForestClassifier(n_estimators=100, random_state=42)
48 rf.fit(X1_train, y1_train)
49
50 # Single test score
51 single_score = rf.score(X1_test, y1_test)
52 print(f"Single train/test split score: {single_score:.3f}")
53
54 # Cross-validation score
55 cv_scores = cross_val_score(rf, X1, y1, cv=5)
56 print(f"5-fold CV scores: {cv_scores}")
57 print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std()})")
58
59 # 2. Different Cross-Validation Strategies
60 print("\n=== DIFFERENT CROSS-VALIDATION STRATEGIES ===")
61
62 # K-Fold Cross-Validation
63 kfold = KFold(n_splits=5, shuffle=True, random_state=42)
64 kfold_scores = cross_val_score(rf, X1, y1, cv=kfold)
65 print(f"K-Fold CV: {kfold_scores.mean():.3f} (+/- {kfold_scores.std()})")
66
67 # Stratified K-Fold Cross-Validation
68 skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
69 skfold_scores = cross_val_score(rf, X1, y1, cv=skfold)
70 print(f"Stratified K-Fold CV: {skfold_scores.mean():.3f} (+/- {skfold_scores.std()})")
71
72 # Leave-One-Out Cross-Validation (slow for large datasets)
73 print("Leave-One-Out CV (using subset for speed):")
74 X1_small = X1[:100] # Use subset for speed
75 y1_small = y1[:100]
76 loocv = LeaveOneOut()
77 loocv_scores = cross_val_score(rf, X1_small, y1_small, cv=loocv)
78 print(f"L00CV: {loocv_scores.mean():.3f} (+/- {loocv_scores.std()})")
79
80 # Shuffle Split Cross-Validation
81 shuffle_split = ShuffleSplit(n_splits=5, test_size=0.2, random_state=42)
82 shuffle_scores = cross_val_score(rf, X1, y1, cv=shuffle_split)
83 print(f"Shuffle Split CV: {shuffle_scores.mean():.3f} (+/- {shuffle_scores.std()})")
84
85 # 3. Cross-Validation for Imbalanced Data
86 print("\n=== CROSS-VALIDATION FOR IMBALANCED DATA ===")
87
88 # Regular K-Fold on imbalanced data
89 kfold_imbalanced = KFold(n_splits=5, shuffle=True, random_state=42)
90 kfold_imb_scores = cross_val_score(rf, X2, y2, cv=kfold_imbalanced)
91 print(f"K-Fold on imbalanced data: {kfold_imb_scores.mean():.3f}")
92
93 # Stratified K-Fold on imbalanced data
94 skfold_imbalanced = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
95 skfold_imb_scores = cross_val_score(rf, X2, y2, cv=skfold_imbalanced)
96 print(f"Stratified K-Fold on imbalanced data: {skfold_imb_scores.mean():.3f}")
97
98 # 4. Multiple Metrics with Cross-Validation
99 print("\n=== MULTIPLE METRICS WITH CROSS-VALIDATION ===")
100
101 # Define scoring metrics
102 scoring = ['accuracy', 'precision', 'recall', 'f1']
103
104 # Cross-validate with multiple metrics
105 cv_results = cross_validate(rf, X1, y1, cv=5, scoring=scoring, return_train_score=True)
106

```

```

111 print("Cross-validation results:")
112 for metric in scoring:
113     test_scores = cv_results[f'test_{metric}']
114     train_scores = cv_results[f'train_{metric}']
115     print(f"{metric.capitalize()}:")
116     print(f"    Test: {test_scores.mean():.3f} (+/- {test_scores.st
117     print(f"    Train: {train_scores.mean():.3f} (+/- {train_scores
118
119 # 5. Cross-Validation Visualization
120 print("\n=== CROSS-VALIDATION VISUALIZATION ===")
121
122 # Plot CV scores for different algorithms
123 algorithms = {
124     'Random Forest': RandomForestClassifier(n_estimators=100, ran
125     'SVM': SVC(random_state=42),
126     'Logistic Regression': LogisticRegression(random_state=42, ma
127     'KNN': KNeighborsClassifier(n_neighbors=5)
128 }
129
130
131 cv_scores_dict = {}
132 for name, model in algorithms.items():
133     scores = cross_val_score(model, X1, y1, cv=5)
134     cv_scores_dict[name] = scores
135     print(f"{name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3
136
137 # Plot CV scores
138 plt.figure(figsize=(12, 6))
139 plt.boxplot([cv_scores_dict[name] for name in algorithms.keys()],
140             labels=list(algorithms.keys()))
141
142 plt.ylabel('CV Score')
143 plt.title('Cross-Validation Scores Comparison')
144 plt.xticks(rotation=45)
145 plt.grid(True)
146 plt.show()
147
148 # 6. Time Series Cross-Validation
149 print("\n=== TIME SERIES CROSS-VALIDATION ===")
150
151 # Create time series data
152 np.random.seed(42)
153 n_samples = 200
154 time_series = np.cumsum(np.random.randn(n_samples)) + np.sin(np.l
155 X_ts = time_series[:-1].reshape(-1, 1)
156 y_ts = (time_series[1:] > time_series[:-1]).astype(int)
157
158 # Time Series Split
159 tscv = TimeSeriesSplit(n_splits=5)
160 tscv_scores = cross_val_score(rf, X_ts, y_ts, cv=tscv)
161 print(f"Time Series CV: {tscv_scores.mean():.3f} (+/- {tscv_score
162
163 # Visualize time series splits
164 plt.figure(figsize=(12, 8))
165 for i, (train_idx, test_idx) in enumerate(tscv.split(X_ts)):
166     plt.subplot(2, 3, i+1)
167     plt.plot(train_idx, y_ts[train_idx], 'b-', label='Train', alp
168     plt.plot(test_idx, y_ts[test_idx], 'r-', label='Test', alpha=
169     plt.title(f'Time Series Split {i+1}')
170     plt.xlabel('Time')
171     plt.ylabel('Value')
172     plt.legend()
173
174

```

```

175 plt.tight_layout()
176 plt.show()
177
178 # 7. Cross-Validation with Preprocessing
179 print("\n=== CROSS-VALIDATION WITH PREPROCESSING ===")
180
181 from sklearn.pipeline import Pipeline
182 from sklearn.preprocessing import StandardScaler
183
184 # Create pipeline with preprocessing
185 pipeline = Pipeline([
186     ('scaler', StandardScaler()),
187     ('classifier', SVC(random_state=42))
188 ])
189
190
191 # Cross-validate pipeline
192 pipeline_scores = cross_val_score(pipeline, X3, y3, cv=5)
193 print(f"Pipeline CV (with scaling): {pipeline_scores.mean():.3f}")
194
195 # Compare with and without scaling
196 rf_no_scale = RandomForestClassifier(n_estimators=100, random_state=42)
197 rf_no_scale_scores = cross_val_score(rf_no_scale, X3, y3, cv=5)
198 print(f"Random Forest (no scaling): {rf_no_scale_scores.mean():.3f}")
199
200 # 8. Cross-Validation for Model Selection
201 print("\n=== CROSS-VALIDATION FOR MODEL SELECTION ===")
202
203 # Compare different models using CV
204 models = {
205     'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
206     'SVM (RBF)': SVC(kernel='rbf', random_state=42),
207     'SVM (Linear)': SVC(kernel='linear', random_state=42),
208     'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000),
209     'KNN (k=3)': KNeighborsClassifier(n_neighbors=3),
210     'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),
211     'KNN (k=7)': KNeighborsClassifier(n_neighbors=7)
212 }
213
214
215 model_scores = {}
216 for name, model in models.items():
217     scores = cross_val_score(model, X1, y1, cv=5)
218     model_scores[name] = scores.mean()
219     print(f"{name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3f})")
220
221
222 # Find best model
223 best_model = max(model_scores, key=model_scores.get)
224 print(f"\nBest model: {best_model} with score: {model_scores[best_model]:.3f}")
225
226 # 9. Cross-Validation Best Practices
227 print("\n=== CROSS-VALIDATION BEST PRACTICES ===")
228
229 print("1. Use stratified CV for classification problems")
230 print("2. Use time series CV for temporal data")
231 print("3. Use appropriate number of folds (5-10)")
232 print("4. Ensure data is shuffled before splitting")
233 print("5. Use same CV strategy for all models")
234 print("6. Consider computational cost")
235 print("7. Use multiple metrics for evaluation")
236 print("8. Validate on holdout set")
237 print("9. Be careful with data leakage")
238

```



```

239     print("10. Document your CV strategy")
240
241     # 10. Cross-Validation Pitfalls
242     print("\n=== CROSS-VALIDATION PITFALLS ===")
243
244     print("Common mistakes to avoid:")
245     print("1. Data leakage: Don't use future data to predict past")
246     print("2. Target leakage: Don't include target-related features")
247     print("3. Inconsistent preprocessing: Apply same preprocessing to")
248     print("4. Wrong CV strategy: Use appropriate CV for your data type")
249     print("5. Overfitting to CV: Don't tune hyperparameters on test set")
250     print("6. Insufficient folds: Too few folds can give unstable estimates")
251     print("7. Not shuffling data: Can lead to biased estimates")
252     print("8. Using test set for model selection: Keep test set separate")
253
254     # 11. Summary
255     print("\n=== CROSS-VALIDATION SUMMARY ===")
256     print("1. Cross-validation provides robust model evaluation")
257     print("2. Different CV strategies for different data types")
258     print("3. Use stratified CV for classification")
259     print("4. Use time series CV for temporal data")
260     print("5. Multiple metrics give better insights")
261     print("6. Avoid data leakage and target leakage")
262     print("7. Use same CV strategy for fair comparison")
263     print("8. Document your CV methodology")
264     print("9. Consider computational cost")
265     print("10. Always validate on holdout set")

```

8. Model Selection and Comparison

What is Model Selection?

Model selection is the process of choosing the best machine learning algorithm and configuration for your specific problem. It's like choosing the right tool for the job - you need to consider the problem type, data characteristics, and performance requirements.

Key Concepts:

- **Algorithm Comparison:** Testing different algorithms on the same data
- **Performance Metrics:** Choosing appropriate metrics for evaluation
- **Statistical Significance:** Determining if differences are meaningful
- **Bias-Variance Tradeoff:** Balancing underfitting and overfitting
- **Cross-Validation:** Robust evaluation methodology

Model Selection Process:

1. **Define Problem:** Classification, regression, clustering, etc.
2. **Prepare Data:** Clean, preprocess, and split data
3. **Select Algorithms:** Choose relevant algorithms
4. **Train Models:** Fit models on training data
5. **Evaluate Performance:** Use appropriate metrics

6. **Compare Results:** Statistical comparison
7. **Select Best Model:** Choose based on criteria
8. **Validate:** Test on holdout set

Evaluation Metrics:

- **Classification:** Accuracy, Precision, Recall, F1-Score, AUC
- **Regression:** MSE, RMSE, MAE, R^2
- **Clustering:** Silhouette Score, Inertia, ARI

Best Practices:

- Use same CV strategy for all models
- Consider multiple metrics
- Test statistical significance
- Consider computational cost
- Document all experiments
- Validate on holdout set

In [...

```
1  # Model Selection and Comparison - Simplified Examples
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from sklearn.datasets import make_classification, make_regression
6  from sklearn.model_selection import train_test_split, cross_val_s
7  from sklearn.ensemble import RandomForestClassifier, RandomForest
8  from sklearn.svm import SVC, SVR
9  from sklearn.linear_model import LogisticRegression, LinearRegres
10 from sklearn.neighbors import KNeighborsClassifier, KNeighborsReg
11 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegr
12 from sklearn.naive_bayes import GaussianNB
13 from sklearn.preprocessing import StandardScaler
14 from sklearn.metrics import (accuracy_score, precision_score, rec
15                               roc_auc_score, mean_squared_error, mea
16
17 from scipy import stats
18 import warnings
19 warnings.filterwarnings('ignore')
20
21 # Create datasets for model comparison
22 print("=== CREATING DATASETS FOR MODEL COMPARISON ===")
23
24 # Dataset 1: Classification
25 X1, y1 = make_classification(n_samples=1000, n_features=10, n_red
26                               n_clusters_per_class=1, random_state=4
27
28 # Dataset 2: Regression
29 X2, y2 = make_regression(n_samples=1000, n_features=10, noise=0.1
30
31 # Dataset 3: Real-world classification
32 cancer = load_breast_cancer()
33 X3, y3 = cancer.data, cancer.target
34
35 # Dataset 4: Real-world regression (using wine dataset for regres
36
37
```

```

38 wine = load_wine()
39 X4, y4 = wine.data, wine.target # Using target as regression pro
40
41 print(f"Dataset 1 (Classification): {X1.shape}, classes: {len(np.
42 print(f"Dataset 2 (Regression): {X2.shape}, target range: {y2.min
43 print(f"Dataset 3 (Cancer): {X3.shape}, classes: {len(np.unique(y
44 print(f"Dataset 4 (Wine): {X4.shape}, target range: {y4.min():.2f
45
46 # Split datasets
47 X1_train, X1_test, y1_train, y1_test = train_test_split(X1, y1, t
48 X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, t
49 X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, t
50 X4_train, X4_test, y4_train, y4_test = train_test_split(X4, y4, t
51
52
53 # Scale data for algorithms that need it
54 scaler1 = StandardScaler()
55 X1_train_scaled = scaler1.fit_transform(X1_train)
56 X1_test_scaled = scaler1.transform(X1_test)
57
58 scaler3 = StandardScaler()
59 X3_train_scaled = scaler3.fit_transform(X3_train)
60 X3_test_scaled = scaler3.transform(X3_test)
61
62 # 1. Classification Model Comparison
63 print("\n=== CLASSIFICATION MODEL COMPARISON ===")
64
65 # Define classification models
66 class_models = {
67     'Random Forest': RandomForestClassifier(n_estimators=100, ran
68     'SVM (RBF)': SVC(kernel='rbf', random_state=42),
69     'SVM (Linear)': SVC(kernel='linear', random_state=42),
70     'Logistic Regression': LogisticRegression(random_state=42, ma
71     'KNN': KNeighborsClassifier(n_neighbors=5),
72     'Decision Tree': DecisionTreeClassifier(random_state=42),
73     'Naive Bayes': GaussianNB(),
74     'Gradient Boosting': GradientBoostingClassifier(random_state=
75 }
76
77 # Compare models using cross-validation
78 print("Cross-validation results (5-fold):")
79 class_results = {}
80 for name, model in class_models.items():
81     if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression
82         scores = cross_val_score(model, X1_train_scaled, y1_train
83     else:
84         scores = cross_val_score(model, X1_train, y1_train, cv=5)
85
86     class_results[name] = scores.mean()
87     print(f"{name}: {scores.mean():.3f} (+/- {scores.std() * 2:.3
88
89 # 2. Regression Model Comparison
90 print("\n=== REGRESSION MODEL COMPARISON ===")
91
92 # Define regression models
93 reg_models = {
94     'Random Forest': RandomForestRegressor(n_estimators=100, rand
95     'SVR (RBF)': SVR(kernel='rbf'),
96     'SVR (Linear)': SVR(kernel='linear'),
97     'Linear Regression': LinearRegression(),
98     'Ridge': Ridge(alpha=1.0),

```

```

102         'Lasso': Lasso(alpha=1.0),
103         'KNN': KNeighborsRegressor(n_neighbors=5),
104         'Decision Tree': DecisionTreeRegressor(random_state=42)
105     }
106
107     # Compare models using cross-validation
108     print("Cross-validation results (5-fold):")
109     reg_results = {}
110     for name, model in reg_models.items():
111         scores = cross_val_score(model, X2_train, y2_train, cv=5, scoring='neg_mean_squared_error')
112         reg_results[name] = -scores.mean() # Convert back to positive MSE
113         print(f"{name}: MSE = {-scores.mean():.3f} (+/- {scores.std():.3f})")
114
115     # 3. Multiple Metrics Comparison
116     print("\n=== MULTIPLE METRICS COMPARISON ===")
117
118     # Define scoring metrics
119     scoring = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']
120
121     # Compare top 3 classification models
122     top_models = ['Random Forest', 'SVM (RBF)', 'Gradient Boosting']
123     print("Multiple metrics comparison:")
124     for name in top_models:
125         model = class_models[name]
126         if name == 'SVM (RBF)':
127             cv_results = cross_validate(model, X1_train_scaled, y1_train, cv=5, scoring=scoring)
128         else:
129             cv_results = cross_validate(model, X1_train, y1_train, cv=5, scoring=scoring)
130
131         print(f"\n{name}:")
132         for metric in scoring:
133             scores = cv_results[f'test_{metric}']
134             print(f"    {metric.capitalize()}: {scores.mean():.3f} (+/- {scores.std():.3f})")
135
136     # 4. Statistical Significance Testing
137     print("\n=== STATISTICAL SIGNIFICANCE TESTING ===")
138
139     # Compare two models using paired t-test
140     model1_scores = cross_val_score(class_models['Random Forest'], X1_train, y1_train, cv=5, scoring='accuracy')
141     model2_scores = cross_val_score(class_models['SVM (RBF)'], X1_train, y1_train, cv=5, scoring='accuracy')
142
143     # Paired t-test
144     t_stat, p_value = stats.ttest_rel(model1_scores, model2_scores)
145     print(f"Random Forest vs SVM (RBF):")
146     print(f"    Random Forest: {model1_scores.mean():.3f} (+/- {model1_scores.std():.3f})")
147     print(f"    SVM (RBF): {model2_scores.mean():.3f} (+/- {model2_scores.std():.3f})")
148     print(f"    t-statistic: {t_stat:.3f}")
149     print(f"    p-value: {p_value:.3f}")
150     print(f"    Significant difference: {'Yes' if p_value < 0.05 else 'No'}")
151
152     # 5. Model Performance Visualization
153     print("\n=== MODEL PERFORMANCE VISUALIZATION ===")
154
155     # Plot classification results
156     plt.figure(figsize=(15, 5))
157
158     plt.subplot(1, 3, 1)
159     models = list(class_results.keys())
160     scores = list(class_results.values())
161     plt.bar(models, scores)

```

```

166 plt.title('Classification Model Comparison')
167 plt.ylabel('CV Score')
168 plt.xticks(rotation=45)
169 plt.grid(True, alpha=0.3)
170
171 # Plot regression results
172 plt.subplot(1, 3, 2)
173 reg_models_list = list(reg_results.keys())
174 reg_scores = list(reg_results.values())
175 plt.bar(reg_models_list, reg_scores)
176 plt.title('Regression Model Comparison')
177 plt.ylabel('MSE')
178 plt.xticks(rotation=45)
179 plt.grid(True, alpha=0.3)
180
181
182 # Plot model complexity vs performance
183 plt.subplot(1, 3, 3)
184 complexity = [1, 2, 3, 4, 5, 6, 7, 8] # Relative complexity
185 performance = list(class_results.values())
186 plt.scatter(complexity, performance, s=100, alpha=0.7)
187 plt.xlabel('Model Complexity')
188 plt.ylabel('Performance')
189 plt.title('Complexity vs Performance')
190 plt.grid(True, alpha=0.3)
191
192 plt.tight_layout()
193 plt.show()
194
195
196 # 6. Learning Curves Comparison
197 print("\n=== LEARNING CURVES COMPARISON ===")
198
199 # Compare learning curves for top models
200 train_sizes = np.linspace(0.1, 1.0, 10)
201 plt.figure(figsize=(12, 8))
202
203 for i, name in enumerate(['Random Forest', 'SVM (RBF)', 'Gradient
204 model = class_models[name]
205 train_scores = []
206 val_scores = []
207
208 for size in train_sizes:
209     n_samples = int(size * len(X1_train))
210     X_subset = X1_train[:n_samples]
211     y_subset = y1_train[:n_samples]
212
213     if name == 'SVM (RBF)':
214         X_subset = scaler1.transform(X_subset)
215
216     # Training score
217     model.fit(X_subset, y_subset)
218     train_score = model.score(X_subset, y_subset)
219     train_scores.append(train_score)
220
221     # Validation score
222     val_score = cross_val_score(model, X_subset, y_subset, cv
223     val_scores.append(val_score)
224
225
226 plt.subplot(2, 2, i+1)
227 plt.plot(train_sizes, train_scores, 'o-', label='Training Sco
228 plt.plot(train_sizes, val_scores, 'o-', label='Validation Sco
229

```

```

230     plt.xlabel('Training Set Size')
231     plt.ylabel('Score')
232     plt.title(f'{name} Learning Curves')
233     plt.legend()
234     plt.grid(True, alpha=0.3)
235
236 plt.tight_layout()
237 plt.show()
238
239 # 7. Model Selection for Different Problem Types
240 print("\n=== MODEL SELECTION FOR DIFFERENT PROBLEM TYPES ===")
241
242 # Small dataset
243 X_small, y_small = X1[:100], y1[:100]
244 print("Small dataset (100 samples):")
245 small_results = {}
246 for name, model in class_models.items():
247     if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression']:
248         scores = cross_val_score(model, scaler1.transform(X_small),
249                                   y_small, cv=5)
250     else:
251         scores = cross_val_score(model, X_small, y_small, cv=3)
252     small_results[name] = scores.mean()
253     print(f" {name}: {scores.mean():.3f}")
254
255 # Large dataset
256 X_large, y_large = X1, y1
257 print("\nLarge dataset (1000 samples):")
258 large_results = {}
259 for name, model in class_models.items():
260     if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression']:
261         scores = cross_val_score(model, X1_train_scaled, y1_train,
262                                   cv=5)
263     else:
264         scores = cross_val_score(model, X1_train, y1_train, cv=5)
265     large_results[name] = scores.mean()
266     print(f" {name}: {scores.mean():.3f}")
267
268 # 8. Real-world Dataset Comparison
269 print("\n=== REAL-WORLD DATASET COMPARISON ===")
270
271 # Compare on breast cancer dataset
272 print("Breast Cancer Dataset:")
273 cancer_results = {}
274 for name, model in class_models.items():
275     if name in ['SVM (RBF)', 'SVM (Linear)', 'Logistic Regression']:
276         scores = cross_val_score(model, X3_train_scaled, y3_train,
277                                   cv=5)
278     else:
279         scores = cross_val_score(model, X3_train, y3_train, cv=5)
280     cancer_results[name] = scores.mean()
281     print(f" {name}: {scores.mean():.3f}")
282
283 # 9. Model Selection Criteria
284 print("\n=== MODEL SELECTION CRITERIA ===")
285
286 # Define selection criteria
287 def select_model(results, criteria='performance'):
288     if criteria == 'performance':
289         return max(results, key=results.get)
290     elif criteria == 'stability':
291         # This would require standard deviations, simplified here
292         return max(results, key=results.get)
293

```

```

294         elif criteria == 'simplicity':
295             # Prefer simpler models
296             simple_models = ['Logistic Regression', 'Naive Bayes', 'D
297             for model in simple_models:
298                 if model in results:
299                     return model
300             return max(results, key=results.get)
301
302     print("Best model by performance:", select_model(class_results, '
303     print("Best model by simplicity:", select_model(class_results, 's
304
305     # 10. Model Selection Best Practices
306     print("\n=== MODEL SELECTION BEST PRACTICES ===")
307
308     print("1. Start with simple models and increase complexity")
309     print("2. Use appropriate evaluation metrics for your problem")
310     print("3. Use cross-validation for robust evaluation")
311     print("4. Test statistical significance of differences")
312     print("5. Consider computational cost and interpretability")
313     print("6. Validate on holdout set")
314     print("7. Consider ensemble methods")
315     print("8. Document all experiments")
316     print("9. Consider business requirements")
317     print("10. Monitor model performance over time")
318
319     # 11. Summary
320     print("\n=== MODEL SELECTION SUMMARY ===")
321     print("1. Model selection is choosing the best algorithm for your
322     print("2. Compare multiple algorithms using cross-validation")
323     print("3. Use appropriate metrics for your problem type")
324     print("4. Test statistical significance of differences")
325     print("5. Consider multiple criteria: performance, complexity, in
326     print("6. Validate on holdout set")
327     print("7. Consider computational cost")
328     print("8. Document all experiments")
329     print("9. Consider ensemble methods")
330     print("10. Monitor performance over time")

```

9. Complete ML Project Workflow

End-to-End Machine Learning Pipeline

Now that we've covered all the individual components, let's put everything together in a complete machine learning project workflow. This section demonstrates how to combine all the techniques we've learned into a real-world project.

Project Workflow Steps:

1. **Problem Definition:** Understand the business problem
2. **Data Collection:** Gather relevant data
3. **Data Exploration:** Understand your data
4. **Data Preprocessing:** Clean and prepare data
5. **Feature Engineering:** Create new features
6. **Model Selection:** Choose appropriate algorithms

7. **Model Training:** Train multiple models
8. **Model Evaluation:** Compare performance
9. **Model Tuning:** Optimize hyperparameters
10. **Model Deployment:** Put model into production
11. **Monitoring:** Track performance over time

Best Practices:

- Start simple and iterate
- Document everything
- Use version control
- Test thoroughly
- Consider business requirements
- Plan for maintenance

In [...

```
1      # Complete ML Project Workflow - End-to-End Example
2
3      import pandas as pd
4      import numpy as np
5      import matplotlib.pyplot as plt
6      import seaborn as sns
7      from sklearn.datasets import load_breast_cancer
8      from sklearn.model_selection import train_test_split, cross_val_score
9      from sklearn.preprocessing import StandardScaler, LabelEncoder
10     from sklearn.ensemble import RandomForestClassifier, VotingClassifier
11     from sklearn.svm import SVC
12     from sklearn.linear_model import LogisticRegression
13     from sklearn.neighbors import KNeighborsClassifier
14     from sklearn.metrics import (classification_report, confusion_matrix,
15                                  roc_curve, precision_recall_curve)
16
17     from sklearn.pipeline import Pipeline
18     from sklearn.compose import ColumnTransformer
19     from sklearn.impute import SimpleImputer
20     from sklearn.feature_selection import SelectKBest, f_classif
21     import joblib
22     import warnings
23     warnings.filterwarnings('ignore')
24
25     # Set random seed for reproducibility
26     np.random.seed(42)
27
28     print("=== COMPLETE MACHINE LEARNING PROJECT WORKFLOW ===")
29     print("Project: Breast Cancer Classification")
30     print("Goal: Predict whether a tumor is malignant or benign")
31
32     # Step 1: Problem Definition
33     print("\n=== STEP 1: PROBLEM DEFINITION ===")
34     print("Business Problem: Early detection of breast cancer")
35     print("Success Metric: High accuracy with good recall (minimize false negatives)")
36     print("Constraints: Model should be interpretable for medical professionals")
37
38     # Step 2: Data Collection
39     print("\n=== STEP 2: DATA COLLECTION ===")
40     cancer = load_breast_cancer()
41     X, y = cancer.data, cancer.target
```

```

43 feature_names = cancer.feature_names
44
45 print(f"Dataset shape: {X.shape}")
46 print(f"Features: {len(feature_names)}")
47 print(f"Classes: {cancer.target_names}")
48 print(f"Class distribution: {np.bincount(y)}")
49
50 # Step 3: Data Exploration
51 print("\n=== STEP 3: DATA EXPLORATION ===")
52
53 # Create DataFrame for easier exploration
54 df = pd.DataFrame(X, columns=feature_names)
55 df['target'] = y
56 df['target_name'] = df['target'].map({0: 'Malignant', 1: 'Benign'})
57
58 print("Dataset Info:")
59 print(df.info())
60
61 print("\nBasic Statistics:")
62 print(df.describe())
63
64 # Visualize class distribution
65 plt.figure(figsize=(12, 4))
66
67 plt.subplot(1, 2, 1)
68 df['target_name'].value_counts().plot(kind='bar')
69 plt.title('Class Distribution')
70 plt.xlabel('Diagnosis')
71 plt.ylabel('Count')
72 plt.xticks(rotation=0)
73
74 # Visualize feature distributions
75 plt.subplot(1, 2, 2)
76 df[feature_names[:5]].boxplot()
77 plt.title('Feature Distributions (First 5 Features)')
78 plt.xticks(rotation=45)
79
80 plt.tight_layout()
81 plt.show()
82
83 # Correlation analysis
84 plt.figure(figsize=(10, 8))
85 correlation_matrix = df[feature_names[:10]].corr()
86 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', cent
87 plt.title('Feature Correlation Matrix (First 10 Features)')
88 plt.show()
89
90 # Step 4: Data Preprocessing
91 print("\n=== STEP 4: DATA PREPROCESSING ===")
92
93 # Check for missing values
94 print(f"Missing values: {df.isnull().sum().sum()}")
95
96 # Check for outliers (using IQR method)
97 def detect_outliers_iqr(data, column):
98     Q1 = data[column].quantile(0.25)
99     Q3 = data[column].quantile(0.75)
100     IQR = Q3 - Q1
101     lower_bound = Q1 - 1.5 * IQR
102     upper_bound = Q3 + 1.5 * IQR

```

```

107         outliers = data[(data[column] < lower_bound) | (data[column]
108             return len(outliers)
109
110     print("Outliers detected (IQR method):")
111     for feature in feature_names[:5]: # Check first 5 features
112         outliers = detect_outliers_iqr(df, feature)
113         print(f" {feature}: {outliers} outliers")
114
115     # Step 5: Feature Engineering
116     print("\n=== STEP 5: FEATURE ENGINEERING ===")
117
118     # Create new features
119     df['mean_radius_squared'] = df['mean radius'] ** 2
120     df['mean_texture_log'] = np.log1p(df['mean texture'])
121     df['area_perimeter_ratio'] = df['mean area'] / (df['mean perimete
122
123
124     # Add engineered features to feature list
125     engineered_features = ['mean_radius_squared', 'mean_texture_log',
126     all_features = list(feature_names) + engineered_features
127
128     print(f"Original features: {len(feature_names)}")
129     print(f"Engineered features: {len(engineered_features)}")
130     print(f"Total features: {len(all_features)}")
131
132     # Step 6: Model Selection
133     print("\n=== STEP 6: MODEL SELECTION ===")
134
135     # Prepare data
136     X_engineered = df[all_features].values
137     y = df['target'].values
138
139     # Split data
140     X_train, X_test, y_train, y_test = train_test_split(X_engineered,
141                                                         random_state=4
142
143
144     print(f"Training set: {X_train.shape}")
145     print(f"Test set: {X_test.shape}")
146
147     # Define models to compare
148     models = {
149         'Random Forest': RandomForestClassifier(n_estimators=100, ran
150         'SVM': SVC(random_state=42),
151         'Logistic Regression': LogisticRegression(random_state=42, ma
152         'KNN': KNeighborsClassifier(n_neighbors=5)
153     }
154
155     # Compare models using cross-validation
156     print("Model comparison (5-fold CV):")
157     model_scores = {}
158     for name, model in models.items():
159         scores = cross_val_score(model, X_train, y_train, cv=5)
160         model_scores[name] = scores.mean()
161         print(f" {name}: {scores.mean():.3f} (+/- {scores.std() * 2:
162
163
164     # Step 7: Model Training
165     print("\n=== STEP 7: MODEL TRAINING ===")
166
167     # Select best model
168     best_model_name = max(model_scores, key=model_scores.get)
169     print(f"Best model: {best_model_name}")
170

```

```

171
172     # Train best model
173     best_model = models[best_model_name]
174     best_model.fit(X_train, y_train)
175
176     # Step 8: Model Evaluation
177     print("\n=== STEP 8: MODEL EVALUATION ===")
178
179     # Make predictions
180     y_pred = best_model.predict(X_test)
181     y_pred_proba = best_model.predict_proba(X_test)[:, 1]
182
183     # Calculate metrics
184     accuracy = best_model.score(X_test, y_test)
185     auc = roc_auc_score(y_test, y_pred_proba)
186
187     print(f"Test Accuracy: {accuracy:.3f}")
188     print(f"Test AUC: {auc:.3f}")
189
190     # Detailed classification report
191     print("\nClassification Report:")
192     print(classification_report(y_test, y_pred, target_names=['Malignant', 'Benign']))
193
194     # Confusion Matrix
195     plt.figure(figsize=(12, 4))
196
197     plt.subplot(1, 2, 1)
198     cm = confusion_matrix(y_test, y_pred)
199     sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
200                 xticklabels=['Malignant', 'Benign'],
201                 yticklabels=['Malignant', 'Benign'])
202     plt.title('Confusion Matrix')
203     plt.ylabel('True Label')
204     plt.xlabel('Predicted Label')
205
206     # ROC Curve
207     plt.subplot(1, 2, 2)
208     fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
209     plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc:.3f})')
210     plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
211     plt.xlabel('False Positive Rate')
212     plt.ylabel('True Positive Rate')
213     plt.title('ROC Curve')
214     plt.legend()
215
216     plt.tight_layout()
217     plt.show()
218
219     # Step 9: Model Tuning
220     print("\n=== STEP 9: MODEL TUNING ===")
221
222     # Hyperparameter tuning for best model
223     if best_model_name == 'Random Forest':
224         param_grid = {
225             'n_estimators': [50, 100, 200],
226             'max_depth': [None, 10, 20],
227             'min_samples_split': [2, 5, 10]
228         }
229         grid_search = GridSearchCV(RandomForestClassifier(random_state=42),
230                                     param_grid, cv=5, scoring='accuracy')
231

```

```

235     grid_search.fit(X_train, y_train)
236
237     print(f"Best parameters: {grid_search.best_params_}")
238     print(f"Best CV score: {grid_search.best_score_:.3f}")
239
240     # Use tuned model
241     tuned_model = grid_search.best_estimator_
242     tuned_accuracy = tuned_model.score(X_test, y_test)
243     print(f"Tuned model accuracy: {tuned_accuracy:.3f}")
244
245 # Step 10: Ensemble Methods
246 print("\n=== STEP 10: ENSEMBLE METHODS ===")
247
248 # Create ensemble model
249 ensemble = VotingClassifier([
250     ('rf', RandomForestClassifier(n_estimators=100, random_state=
251     ('svm', SVC(probability=True, random_state=42)),
252     ('lr', LogisticRegression(random_state=42, max_iter=1000))
253 ], voting='soft')
254
255 # Train ensemble
256 ensemble.fit(X_train, y_train)
257 ensemble_accuracy = ensemble.score(X_test, y_test)
258 ensemble_auc = roc_auc_score(y_test, ensemble.predict_proba(X_test)[:, 1])
259
260 print(f"Ensemble Accuracy: {ensemble_accuracy:.3f}")
261 print(f"Ensemble AUC: {ensemble_auc:.3f}")
262
263 # Step 11: Feature Importance
264 print("\n=== STEP 11: FEATURE IMPORTANCE ===")
265
266 # Get feature importance from Random Forest
267 if hasattr(best_model, 'feature_importances_'):
268     feature_importance = pd.DataFrame({
269         'feature': all_features,
270         'importance': best_model.feature_importances_
271     }).sort_values('importance', ascending=False)
272
273 print("Top 10 Most Important Features:")
274 print(feature_importance.head(10))
275
276 # Plot feature importance
277 plt.figure(figsize=(10, 6))
278 top_features = feature_importance.head(10)
279 plt.barh(range(len(top_features)), top_features['importance'])
280 plt.yticks(range(len(top_features)), top_features['feature'])
281 plt.xlabel('Feature Importance')
282 plt.title('Top 10 Most Important Features')
283 plt.gca().invert_yaxis()
284 plt.tight_layout()
285 plt.show()
286
287 # Step 12: Model Pipeline
288 print("\n=== STEP 12: MODEL PIPELINE ===")
289
290 # Create complete pipeline
291 pipeline = Pipeline([
292     ('scaler', StandardScaler()),
293     ('feature_selector', SelectKBest(score_func=f_classif, k=10)),
294     ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))
295 ])

```

```

299     ])
300
301     # Train pipeline
302     pipeline.fit(X_train, y_train)
303     pipeline_accuracy = pipeline.score(X_test, y_test)
304
305     print(f"Pipeline Accuracy: {pipeline_accuracy:.3f}")
306
307     # Step 13: Model Persistence
308     print("\n=== STEP 13: MODEL PERSISTENCE ===")
309
310     # Save model
311     joblib.dump(best_model, 'breast_cancer_model.pkl')
312     print("Model saved as 'breast_cancer_model.pkl'")
313
314     # Load model
315     loaded_model = joblib.load('breast_cancer_model.pkl')
316     loaded_accuracy = loaded_model.score(X_test, y_test)
317     print(f"Loaded model accuracy: {loaded_accuracy:.3f}")
318
319     # Step 14: Model Validation
320     print("\n=== STEP 14: MODEL VALIDATION ===")
321
322     # Validate on different data splits
323     from sklearn.model_selection import StratifiedKFold
324
325     skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
326     cv_scores = cross_val_score(best_model, X_engineered, y, cv=skf)
327
328     print(f"Cross-validation scores: {cv_scores}")
329     print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std()})")
330
331     # Step 15: Business Impact Analysis
332     print("\n=== STEP 15: BUSINESS IMPACT ANALYSIS ===")
333
334     # Calculate business metrics
335     tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
336
337     print("Business Impact Metrics:")
338     print(f"True Negatives (Correctly identified benign): {tn}")
339     print(f"False Positives (Incorrectly identified as malignant): {fp}")
340     print(f"False Negatives (Missed malignant cases): {fn}")
341     print(f"True Positives (Correctly identified malignant): {tp}")
342
343     print(f"\nSensitivity (Recall): {tp/(tp+fn):.3f}")
344     print(f"Specificity: {tn/(tn+fp):.3f}")
345     print(f"Precision: {tp/(tp+fp):.3f}")
346
347     # Step 16: Model Monitoring
348     print("\n=== STEP 16: MODEL MONITORING ===")
349
350     # Simulate model performance over time
351     np.random.seed(42)
352     months = range(1, 13)
353     performance = [accuracy] * 12 + np.random.normal(0, 0.01, 12)
354
355     plt.figure(figsize=(10, 6))
356     plt.plot(months, performance, 'o-')
357     plt.axhline(y=accuracy, color='r', linestyle='--', label='Baseline')
358     plt.xlabel('Month')

```

```

363 plt.ylabel('Accuracy')
364 plt.title('Model Performance Over Time')
365 plt.legend()
366 plt.grid(True, alpha=0.3)
367 plt.show()
368
369 # Step 17: Project Summary
370 print("\n=== STEP 17: PROJECT SUMMARY ===")
371
372 print("Project Results:")
373 print(f"  Best Model: {best_model_name}")
374 print(f"  Test Accuracy: {accuracy:.3f}")
375 print(f"  Test AUC: {auc:.3f}")
print(f"  Cross-validation Score: {cv_scores.mean():.3f}")

print("\nKey Insights:")
print("  1. Random Forest performed best for this dataset")
print("  2. Feature engineering improved model performance")
print("  3. Model shows good sensitivity for cancer detection")
print("  4. Ensemble methods provided additional robustness")

print("\nNext Steps:")
print("  1. Deploy model to production")
print("  2. Set up monitoring system")
print("  3. Collect feedback from medical professionals")
print("  4. Retrain model with new data")
print("  5. Consider model interpretability improvements")

# Clean up
import os
if os.path.exists('breast_cancer_model.pkl'):
    os.remove('breast_cancer_model.pkl')
    print("\nCleared up temporary files")

print("\n=== PROJECT COMPLETED SUCCESSFULLY! ===")

```